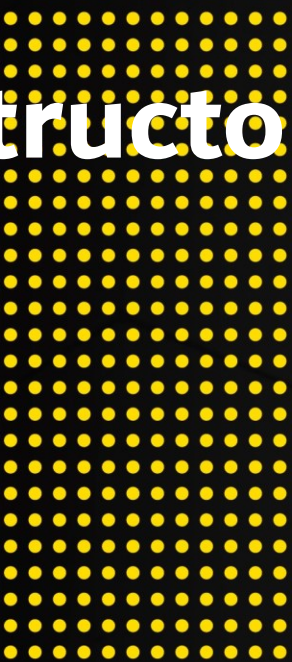




Analítica de Datos con IA, énfasis en energías renovables

UNIV. DE LOS ANDES

2025



Instructor

Juan Carlos Vega, MsC

Geólogo de la UIS, Master en Ciencias de la Tierra de la U EAFIT de Medellín. Especialista en I.A.

Geofísico con más de 15 años de experiencia en O&G. Ha trabajado en evaluación de proyectos de Petróleo y Gas, Minería y licencias ambientales; además es consultor en proyectos de agrícolas y forestales a partir de análisis con inteligencia artificial.

Actualmente coordinador de proyectos para energías renovables y machine learning del grupo de investigación IGGI de la Universidad Industrial de Santander, Colombia y consultor independiente en geofísica ambiental.

EDA

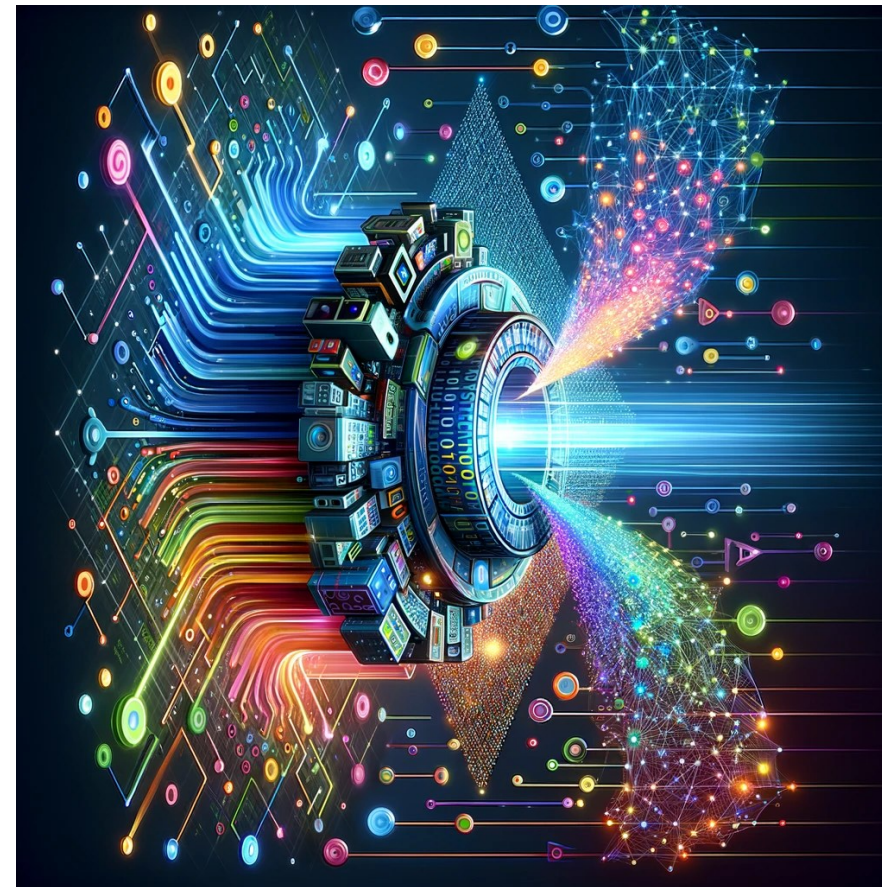
Alistamiento de datos



Que es el pre-procesamiento de datos en Machine learning

Conceptos

- Analisis descriptivo
- Visualización
- Imputación de datos faltantes
- Escalamiento
- Outliers
- Analisis de correlación
- PCA
- Transformación de variables
- Feature engineering



Imputación de datos faltantes

Conceptos



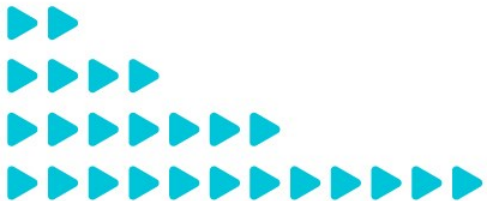
Por diversas razones, muchos conjuntos de datos del mundo real contienen valores faltantes, a menudo codificados como espacios en blanco, NaN u otros marcadores de posición. Sin embargo, dichos conjuntos de datos son incompatibles con los estimadores de scikit-learn que suponen que todos los valores de una matriz son numéricos y que todos tienen y mantienen un significado.

Imputación por la Media/Mediana/Moda:

Conceptos

Esta técnica implica reemplazar los datos faltantes por la media (para datos numéricos), mediana (para datos numéricos con outliers) o moda (para datos categóricos) de la columna.

Cuándo usar: Es efectiva para conjuntos de datos con una cantidad pequeña de observaciones faltantes y cuando la distribución de los datos no es muy sesgada. No recomendada para datos con muchos valores faltantes o cuando los datos faltantes no son aleatorios



Imputación por K-Vecinos más Cercanos (KNN):

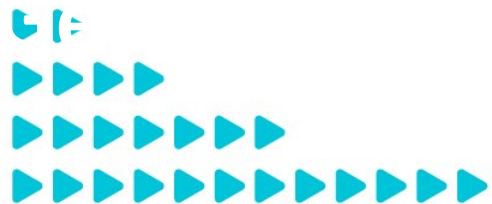
Conceptos



Utiliza los K vecinos más cercanos encontrados en el conjunto de datos para imputar los valores faltantes, basándose en la similitud de las características.



Cuándo usar: Útil cuando los datos muestran patrones o agrupaciones. No es ideal para conjuntos de datos muy grandes debido a su naturaleza computacionalmente costosa.



Imputación por Regresión

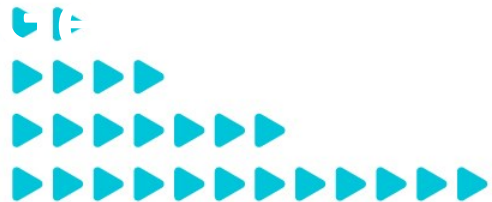
Conceptos



Implica el uso de modelos de regresión para predecir y rellenar los datos faltantes basándose en las relaciones entre variables.



Cuándo usar: Cuando existe una correlación fuerte entre las variables. No es recomendable si las relaciones entre variables son débiles o no lineales sin transformaciones adecuadas.



Imputación Múltiple

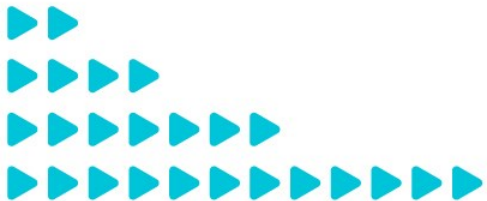
Conceptos



Genera múltiples imputaciones para los valores faltantes utilizando modelos que tienen en cuenta la incertidumbre de la imputación.

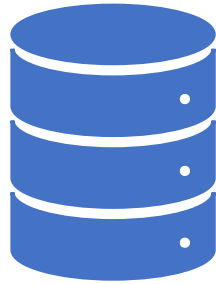


Cuándo usar: En análisis más sofisticados donde es crucial estimar la incertidumbre de las imputaciones. Adecuada para investigaciones científicas o estudios detallados.



Escalamiento

Conceptos



Algoritmos basados en árboles de decisión o de arquitectura similar son insensibles a la escala de los datos, mientras que las ANN dependen del escalamiento y la normalización para funcionar de manera óptima.



Las dos técnicas de escala más utilizadas son la normalización máxima-mínima y la estandarización.

$$x'_i = \frac{x_i - x_{min}}{x_{max}}, \quad i = 1, \dots, n$$

$$x'_i = \frac{x_i - \bar{x}}{\sigma}, \quad i = 1, \dots, n$$



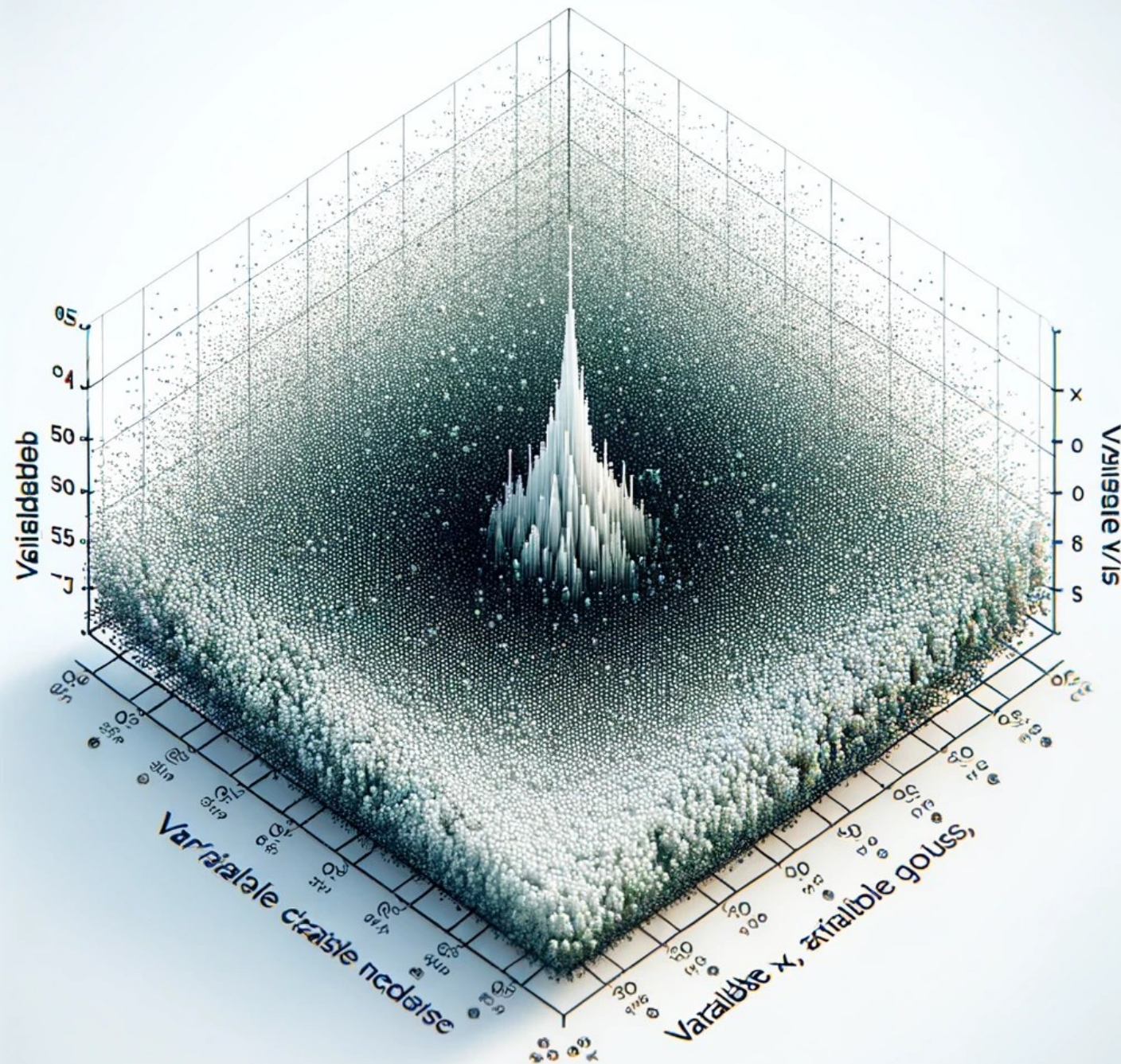
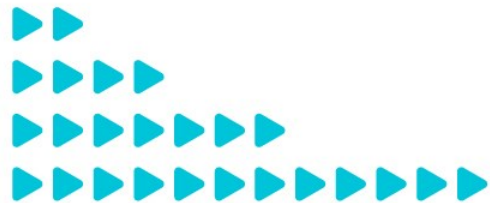
Valores atípicos

Conceptos

En el contexto del Análisis Exploratorio de Datos (EDA), los datos atípicos o outliers son observaciones que se desvían de manera significativa del resto de los datos. Estas observaciones pueden parecer inusuales comparadas con el conjunto de datos general debido a su magnitud, valor o comportamiento.

Los hay extremadamente altos o bajos

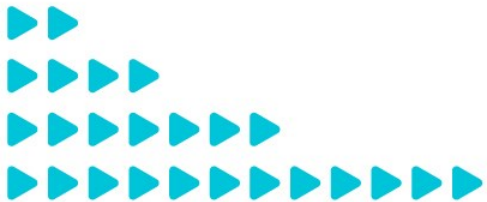
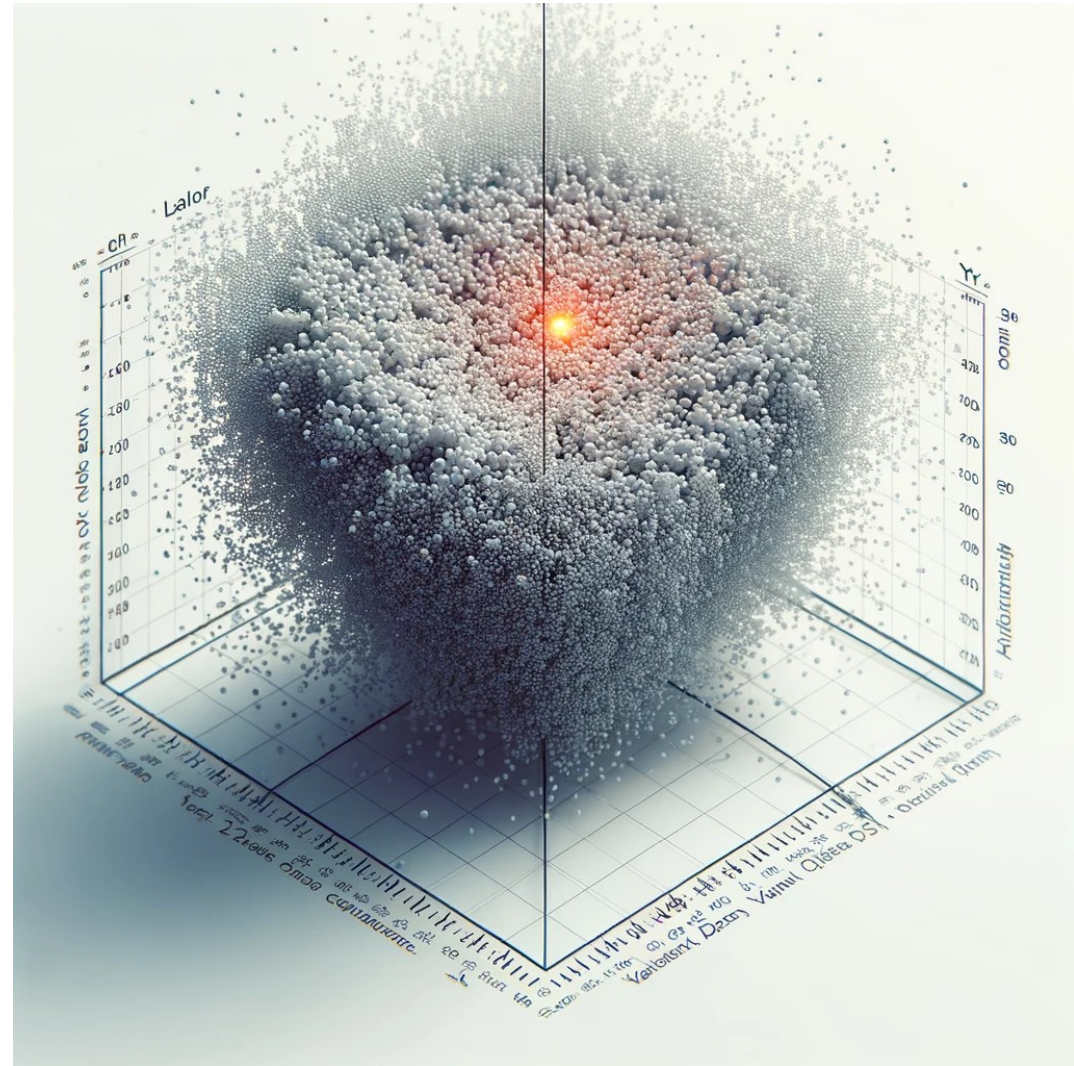
No conformes con el valor esperado



Detección de Valores atípicos

Conceptos

- Analisis visual.
- Estadísticas descriptivas
- IQR (Rango intercuartilo)
- Pruebas estadísticas

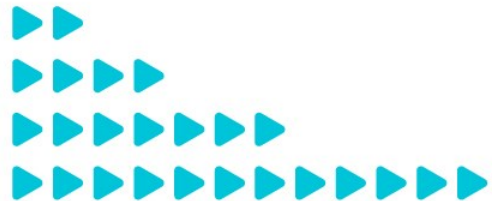


Identificación de Valores atípicos

Conceptos

Una vez detectados, es importante determinar si los valores atípicos son errores de medición o si representan variaciones válidas dentro de los datos.

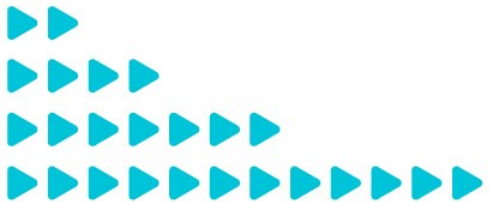
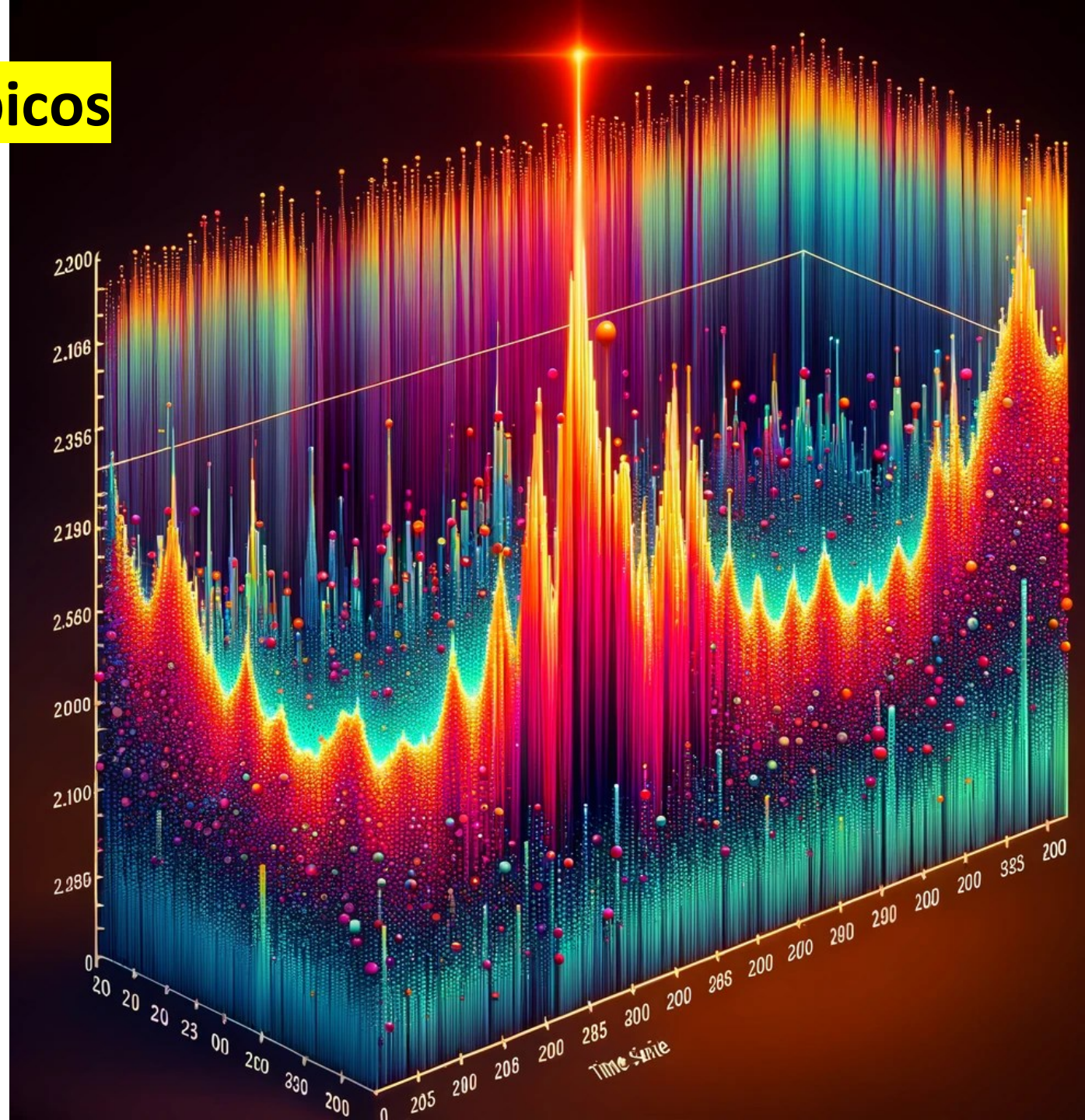
Esto puede requerir conocimiento del dominio y, en ocasiones, una investigación adicional para entender la causa de estos valores extremos.



Tratamiento de Valores atípicos

Conceptos

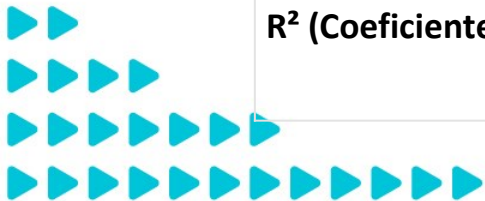
- Eliminación.
- Transformación.
- Imputación.
- Limitación.



Técnicas de evaluación de precisión en ML

O análisis de error

Métrica	Definición	Uso
R (Coeficiente de Correlación de Pearson)	Mide la correlación lineal entre dos variables. Varía entre -1 y 1, donde 1 significa una correlación positiva perfecta, -1 una correlación negativa perfecta, y 0 ninguna correlación.	Utilizado para evaluar la fuerza y la dirección de la relación lineal entre variables.
MSE (Mean Squared Error)	Es el promedio de los cuadrados de los errores. El error es la diferencia entre los valores observados y los predichos.	Utilizado para medir la calidad de un estimador; es siempre no negativo, y valores más bajos indican un mejor ajuste.
RMSE (Root Mean Squared Error)	Es la raíz cuadrada del MSE. Proporciona una medida de la magnitud del error en las mismas unidades que la variable de respuesta.	Utilizado para medir la diferencia entre valores predichos por un modelo y los valores observados.
MAE (Mean Absolute Error)	Es el promedio de los valores absolutos de los errores.	Utilizado para medir cuánto difieren las predicciones de un modelo de los valores reales. Es menos sensible a los outliers que el MSE.
R² (Coeficiente de Determinación)	Proporciona una medida de cuánto de la variabilidad de una variable dependiente es explicada por el modelo. Varía entre 0 y 1.	Utilizado para evaluar la bondad de ajuste de un modelo lineal. Cuanto más cercano a 1, mejor explica el modelo la variabilidad de los datos.



Error Cuadrático medio(RMSE)

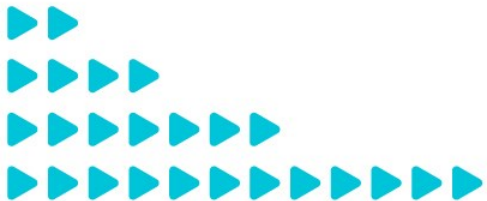
¿Como funciona?

La métrica más comúnmente utilizada para las tareas de regresión es el error cuadrático medio y representa a la raíz cuadrada de la distancia cuadrada promedio entre el valor real y el valor pronosticado. El error cuadrático medio o RMSE es una medida absoluta de ajuste.

$$RMSE = \sqrt{\frac{1}{n} \sum_{j=1}^n (y_j - \hat{y}_j)^2}$$

Indica el ajuste absoluto del modelo a los datos, cuán cerca están los puntos de datos observados de los valores predichos del modelo.

Los valores más bajos de RMSE indican un mejor ajuste. RMSE es una buena medida de la precisión con que el modelo predice la respuesta, y es el criterio más importante para ajustar si el propósito principal del modelo es la predicción.

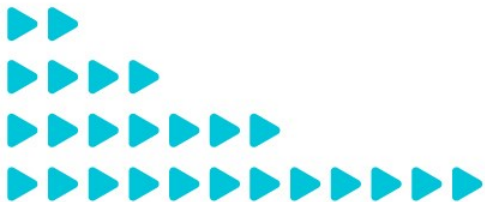


Error Absoluto medio(MAE)

¿Como funciona?

Es el promedio de la diferencia absoluta entre el valor observado y los valores predichos. El error absoluto medio o MAE es un puntaje lineal, lo que significa que todas las diferencias individuales se ponderan por igual en el promedio. Por ejemplo, la diferencia entre 10 y 0 será el doble de la diferencia entre 5 y 0.

$$MAE = \frac{1}{n} \sum_{j=1}^n |y_j - \hat{y}_j|$$



R²

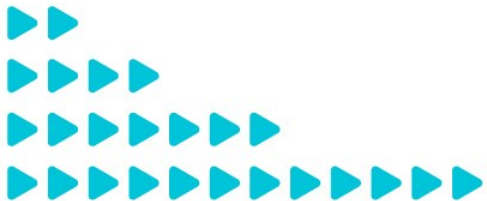
¿Como funciona?

R-cuadrado indica la bondad o la aptitud del modelo, a menudo se utiliza con fines descriptivos y muestra que también las variables independientes seleccionadas explican la variabilidad en sus variables dependiente.

R-cuadrado tiene la propiedad útil de que su escala es intuitiva, va de 0 a 1, con 0 indicando que el modelo propuesto no mejora la predicción sobre el modelo medio y 1 indica una predicción perfecta. La mejora en el modelo de regresión da como resultado aumentos proporcionales en R-cuadrado.

Una de las dificultades de R-cuadrado es que solo puede aumentar a medida que se agregan predictores al modelo de regresión. Este aumento es artificial cuando los predictores no mejoran el ajuste del modelo. Para remediar esto, una estadística relacionada es R-cuadrado ajustado, incorpora los grados de libertad del modelo. El R-cuadrado ajustado disminuirá a medida que agreguen predictores si el aumento en el ajuste del modelo no compensa las pérdidas de grados de libertad. Asimismo, aumentará a medida que se agreguen predictores si vale la pena el aumento en el ajuste del modelo. El R-cuadrado ajustado siempre se debe usar con modelos con más de una variable de predicción.

$$R^2 = 1 - \frac{\sum (y_j - \hat{y}_j)^2}{\sum (y_j - \bar{y}_j)^2}$$

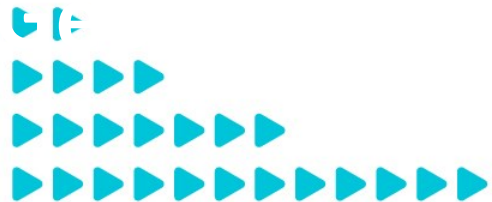


Diferencias entre métricas de evaluación

¿Como funcionan?

El error cuadrático medio (MSE) y el error cuadrático medio penalizan los grandes errores de predicción frente al error absoluto medio (MAE). Sin embargo, RMSE se usa ampliamente que MSE para evaluar el desempeño del modelo de regresión con otros modelos aleatorios, ya que tiene las mismas unidades que la variable dependiente (eje Y).

MSE es una función diferenciable que facilita la realización de operaciones matemáticas en comparación con una función no diferenciable como MAE. Por lo tanto, en muchos modelos, RMSE se utiliza como métrica predeterminada para calcular la función de pérdida a pesar de ser más difícil de interpretar que MAE.



Diferencias entre métricas de evaluación

¿Como funcionan?

El valor más bajo de MAE, MSE y RMSE implica una mayor precisión de un modelo de regresión. Sin embargo, se considera deseable un valor más alto de R cuadrado.

R cuadrado y R cuadrado ajustado se utilizan para explicar qué tan bien las variables independientes en el modelo de regresión lineal explican la variabilidad en la variable dependiente. El valor de R cuadrado siempre aumenta con la suma de las variables independientes, lo que podría llevar a la suma de variables redundantes en nuestro modelo. Sin embargo, el R cuadrado ajustado resuelve este problema.



Diferencias entre métricas de evaluación

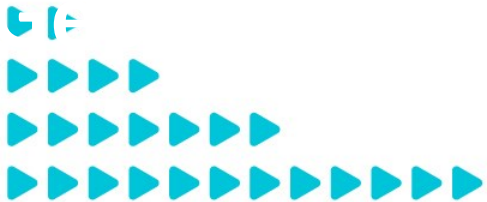
¿Como funcionan?



El R cuadrado ajustado tiene en cuenta el número de variables predictivas y se utiliza para determinar el número de variables independientes en nuestro modelo. El valor de R cuadrado ajustado disminuye si el aumento en el R cuadrado por la variable adicional no es lo suficientemente significativo.



Para comparar la precisión entre diferentes modelos de regresión lineal, RMSE es una mejor opción que R cuadrado.



Regresión y series de tiempo

Técnicas y algoritmos



Series de tiempo

Conceptos



ARIMA (Autoregressive Integrated Moving Average): Modelo clásico para analizar y predecir series temporales lineales, especialmente útil cuando los datos muestran patrones claros de tendencia o estacionalidad.



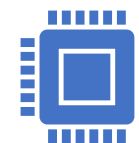
SARIMA (Seasonal ARIMA): Extensión de ARIMA que incorpora componentes estacionales, ideal para datos con patrones estacionales.



Prophet: Desarrollado por Facebook, es un modelo que maneja series temporales con tendencias y patrones estacionales fuertes, incluyendo días festivos y cambios de tendencia.



Modelos de Redes Neuronales Recurrentes (RNN): Especialmente las LSTM (Long Short-Term Memory) y GRU (Gated Recurrent Units), son poderosas para modelar secuencias temporales con dependencias a largo plazo.

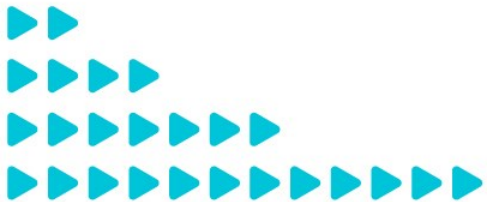
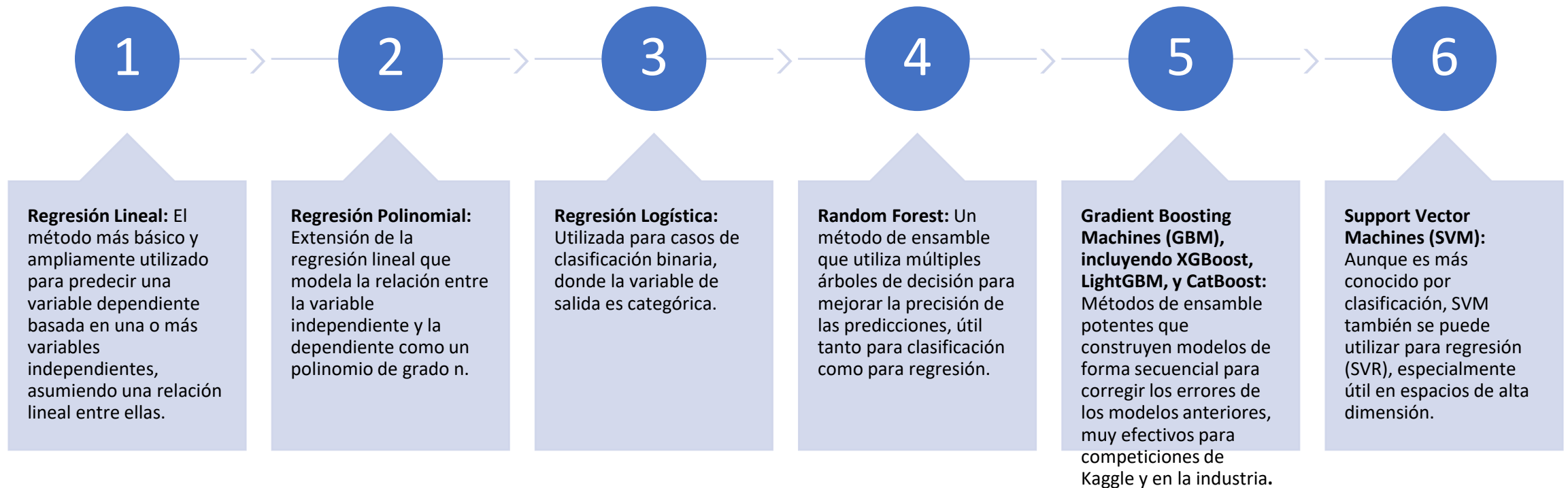


Modelos de Aprendizaje Profundo: Como las Redes Neuronales Convolucionales (CNN) aplicadas a series temporales y los Transformers, que han mostrado resultados prometedores en el análisis de series temporales complejas.



Regresiones

Conceptos



Regresión logística

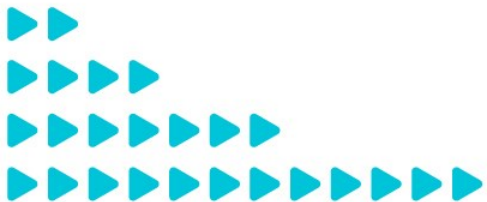
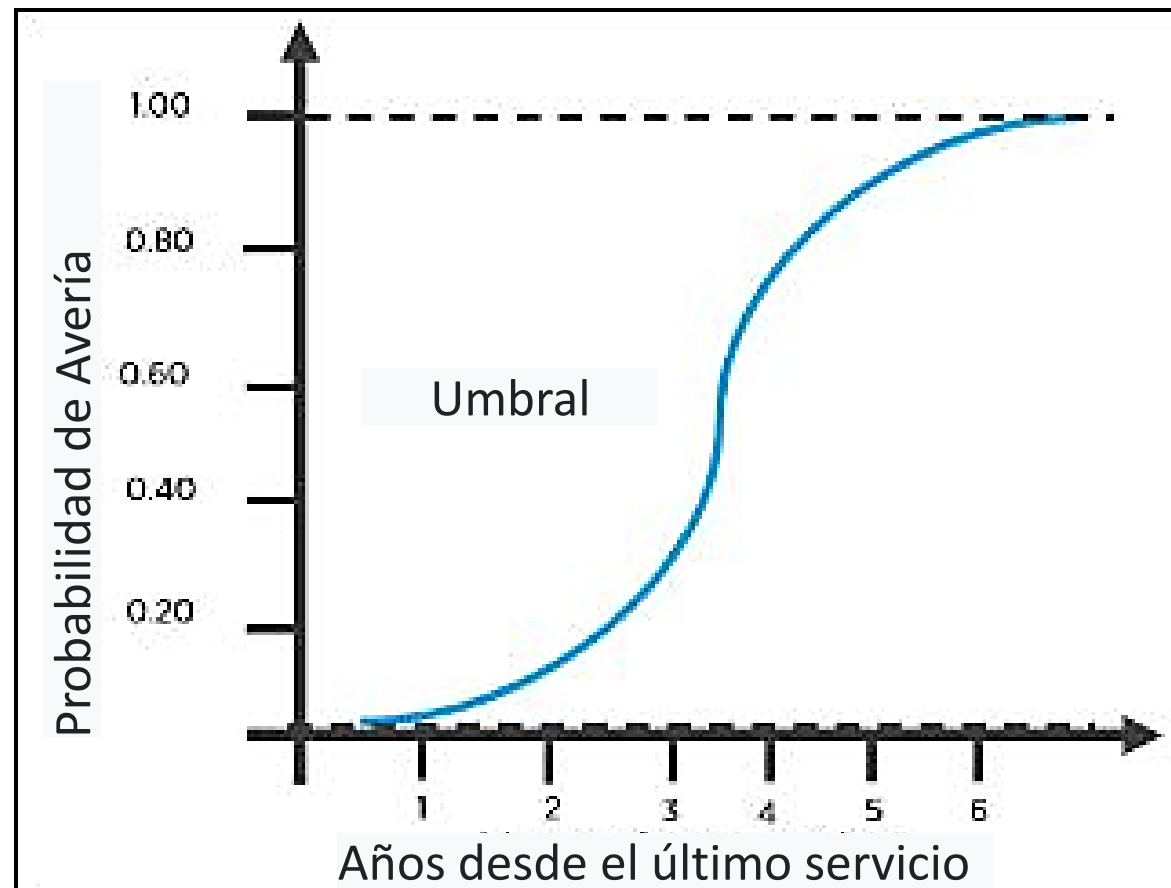
Conceptos



La regresión logística es un método estadístico que se utiliza para construir modelos de aprendizaje automático donde la variable dependiente es dicotómica: es decir, binaria. La regresión logística se utiliza para describir datos y la relación entre una variable dependiente y una o más variables independientes. Las variables independientes pueden ser nominales, ordinales o de tipo intervalo.

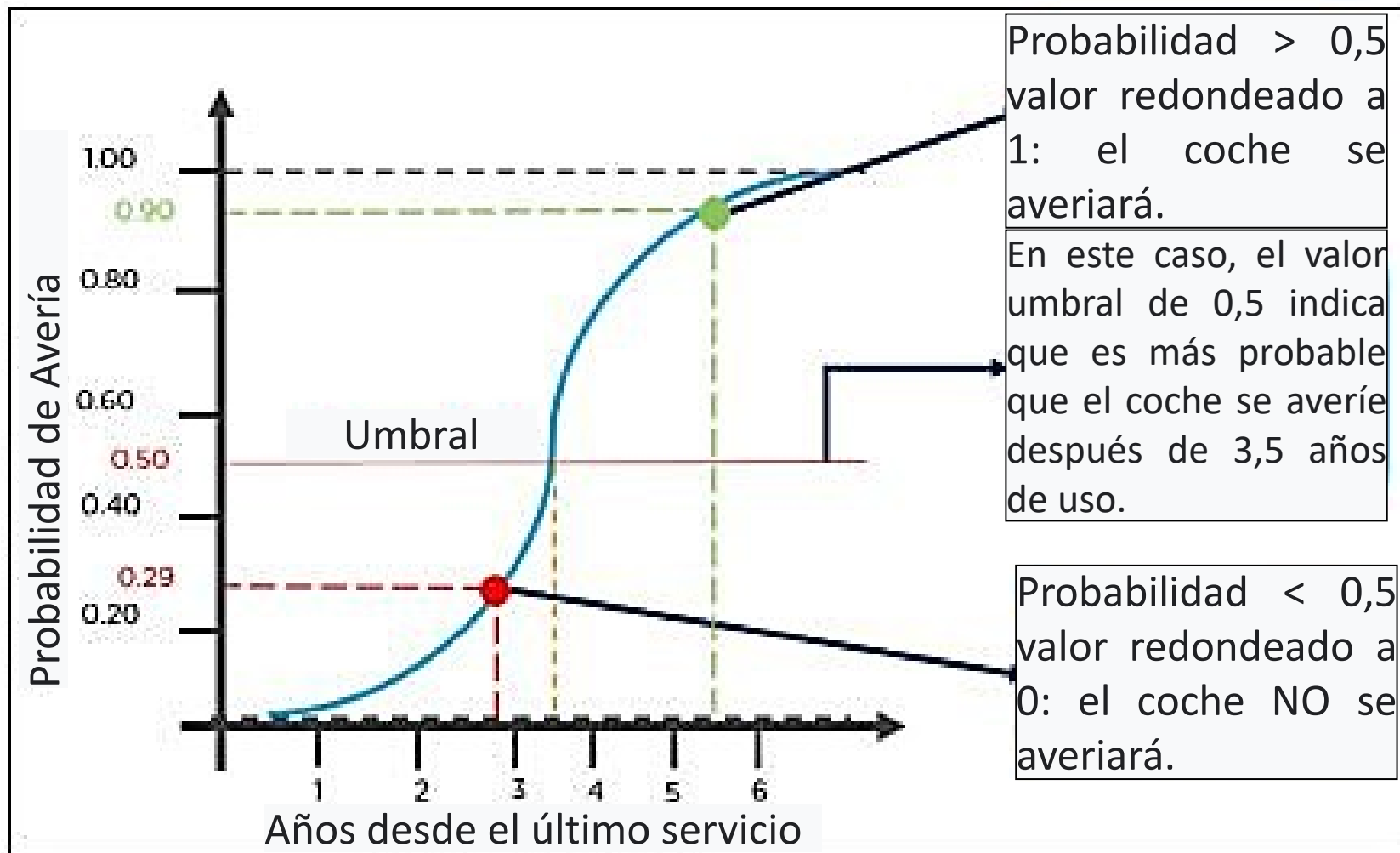


El nombre "regresión logística" se deriva del concepto de función logística que utiliza. La función logística también se conoce como función sigmoidea. El valor de esta función logística se encuentra entre cero y uno.



Regresión logística

Conceptos



Regresión logística

Ventajas



La regresión logística funciona mejor cuando los datos son linealmente separables.



No requiere demasiados recursos computacionales ya que es altamente interpretable.



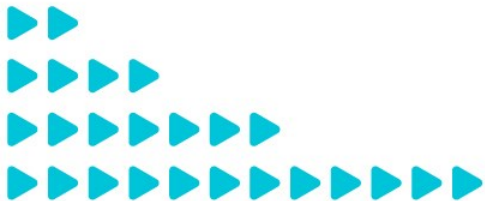
No hay problema para escalar las funciones de entrada: no requiere ajuste.



Es fácil implementar y entrenar un modelo mediante regresión logística.



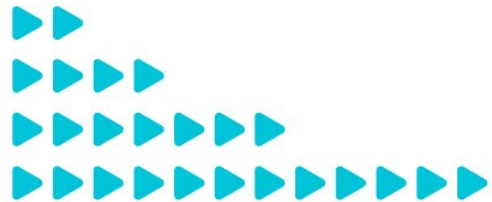
Proporciona una medida de la relevancia de un predictor (tamaño del coeficiente) y su dirección de asociación (positiva o negativa).



Regresión logística

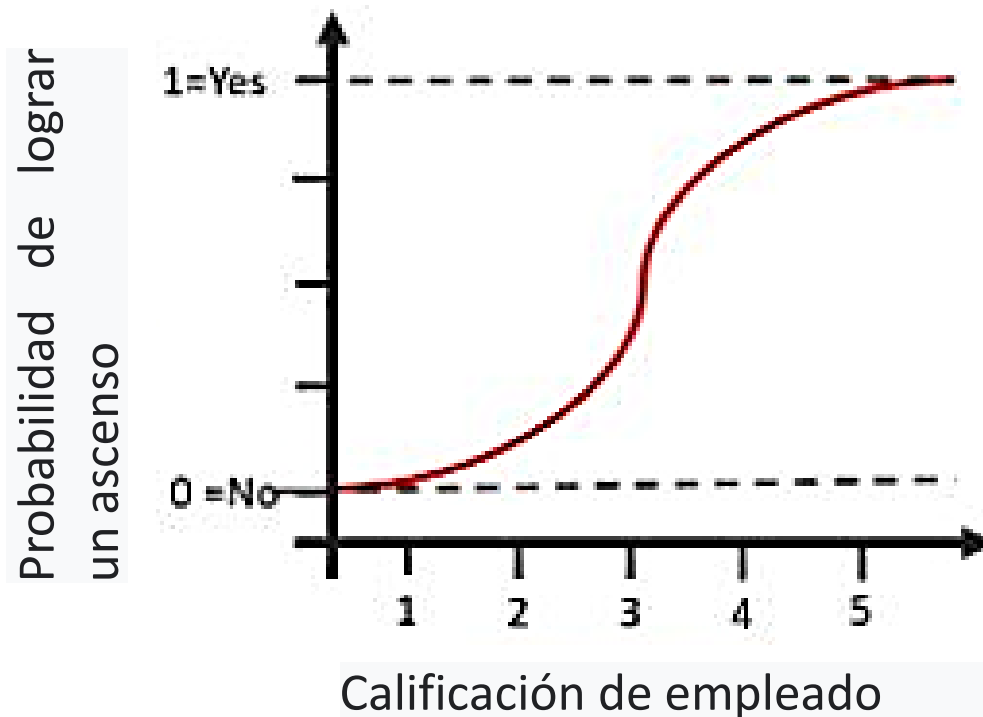
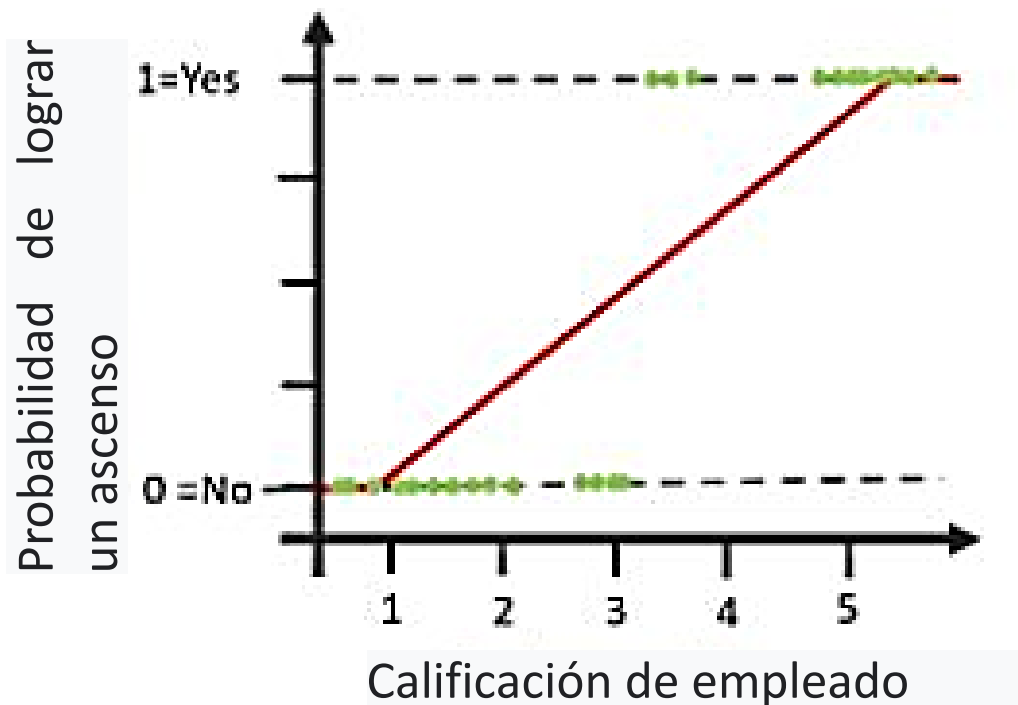
¿Como funciona?

Consideremos una organización quiere determinar el aumento salarial de un empleado en función de su desempeño.



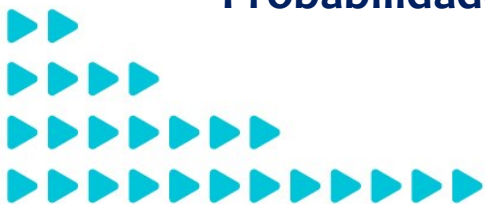
Regresión logística

¿Como funciona?



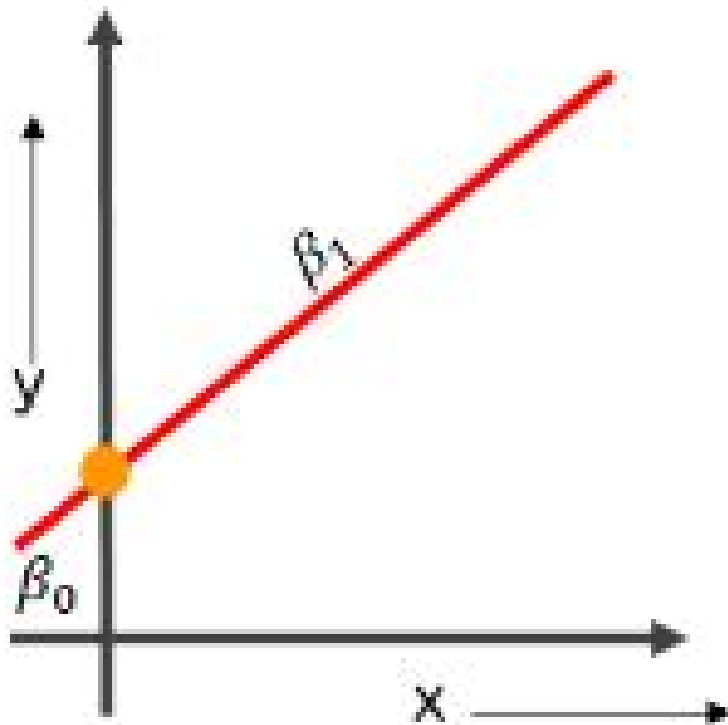
Probabilidades (θ) = Probabilidad de que ocurra un evento / Probabilidad de que no suceda un evento

$$\theta = p / 1 - p$$



Regresión logística

¿Como funciona?



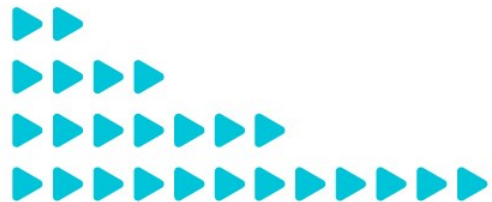
Para predecir las probabilidades de éxito, utilizamos la siguiente fórmula:

$$\log \left(\frac{p(x)}{1-p(x)} \right) = \beta_0 + \beta_1 x$$

$$e^{\ln \left(\frac{p(x)}{1-p(x)} \right)} = e^{\beta_0 + \beta_1 x}$$

$$\left(\frac{p(x)}{1-p(x)} \right) = e^{\beta_0 + \beta_1 x}$$

$$p(x) = \frac{e^{\beta_0 + \beta_1 x}}{1 + e^{\beta_0 + \beta_1 x}}$$

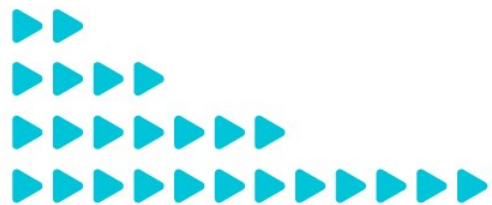
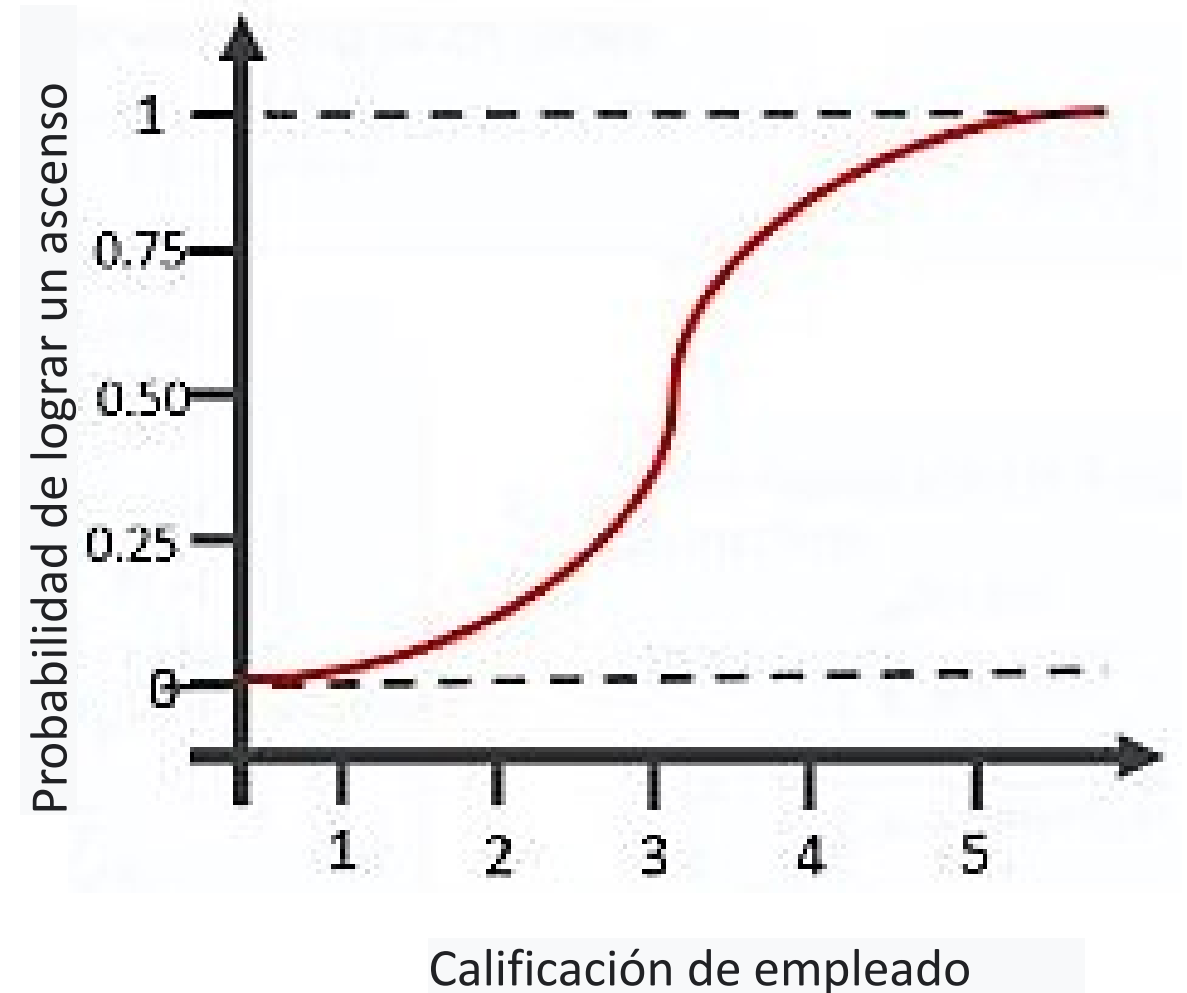


Regresión logística

¿Cómo funciona?

La ecuación de la función sigmoidea es

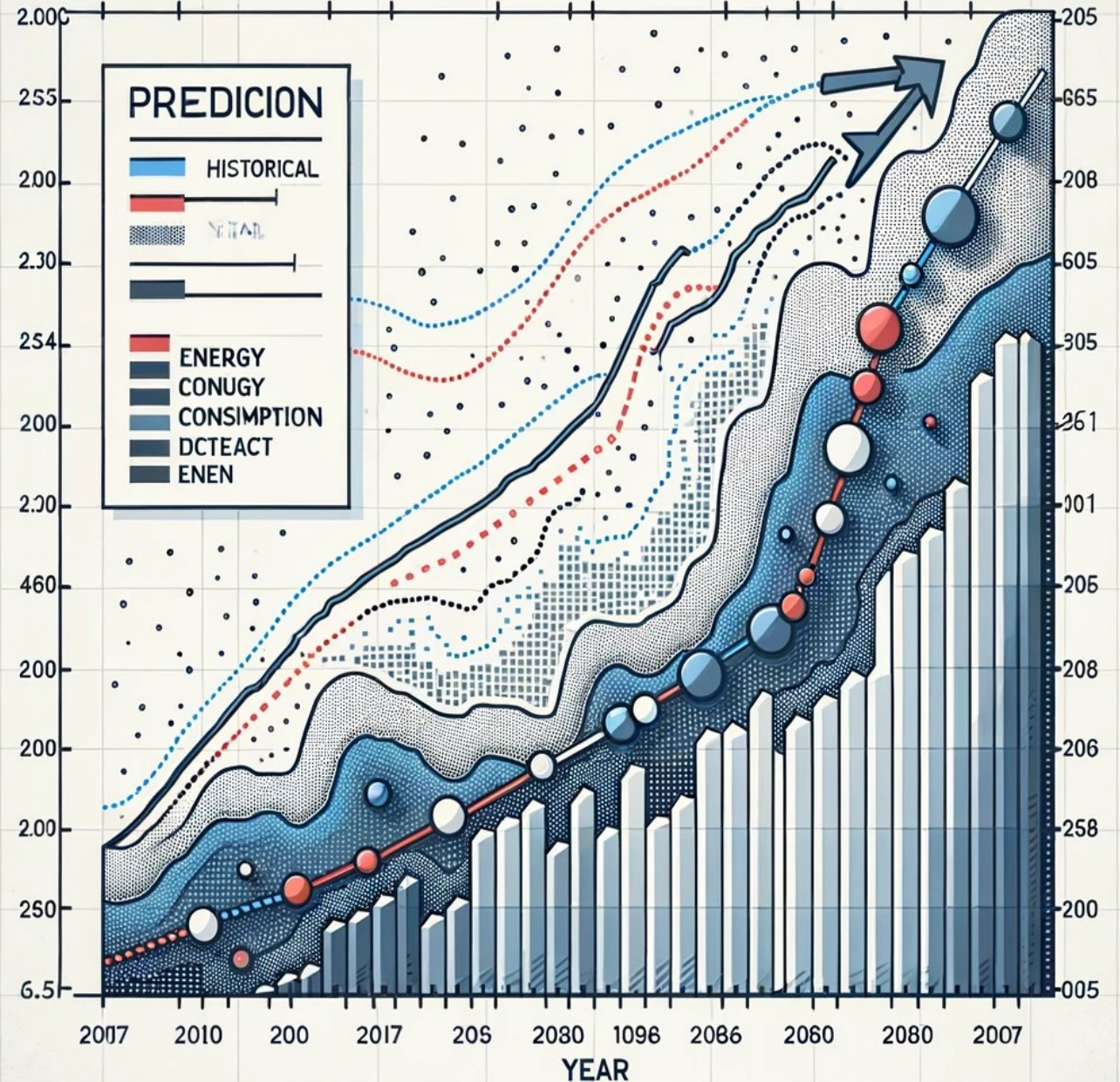
$$p(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$



Elección del método

Conceptos

- La naturaleza de los datos (linealidad, estacionalidad, tendencia, etc.).
- El tamaño y la calidad del conjunto de datos.
- Los recursos computacionales disponibles.
- El objetivo del análisis o la predicción.



Elección del método

Conceptos

Técnica	Tipo	Descripción	Aplicaciones
LSTM	Red neuronal	Una arquitectura de red neuronal recurrente diseñada para aprender dependencias a largo plazo. Es especialmente útil para secuencias temporales.	Predicción de series temporales, procesamiento de lenguaje natural, reconocimiento de voz.
Regresión Logística	Estadística/Machine Learning	Un modelo de regresión utilizado para predecir la probabilidad de una variable categórica binaria.	Clasificación binaria, como determinar si un email es spam o no.
Regresión por Random Forest	Machine Learning	Un método de ensamble que utiliza múltiples árboles de decisión para realizar predicciones.	Predicción de variables continuas (regresión) y clasificación.
ARIMA	Series temporales	Modelo lineal que utiliza diferenciación para hacer la serie temporal estacionaria y luego aplica un modelo autoregresivo y de promedio móvil.	Predicción de datos de series temporales en finanzas, economía, y otros campos.
ARIMAX	Series temporales	Extensión del modelo ARIMA que incluye variables exógenas para mejorar las predicciones.	Modelado de series temporales donde se conocen factores externos que influyen en la variable de interés.



EDA

Ingeniería de Características



Ingeniería de características

Conceptos



La **ingeniería de características** es el proceso de transformar datos sin procesar en características significativas que los algoritmos de aprendizaje automático pueden utilizar para realizar predicciones precisas. Implica seleccionar, extraer y transformar características para mejorar el rendimiento del modelo. Una buena ingeniería de funciones puede marcar una gran diferencia en la precisión del modelo, mientras que una mala ingeniería de funciones puede provocar un rendimiento deficiente.

IC en Machine learning

Conceptos

Selección y extracción de características

Codificación de variables categóricas

Escalado y normalización

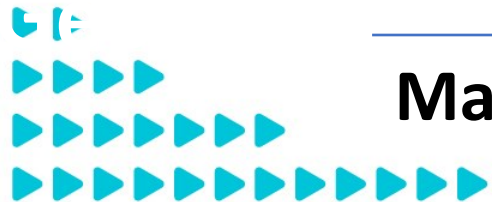
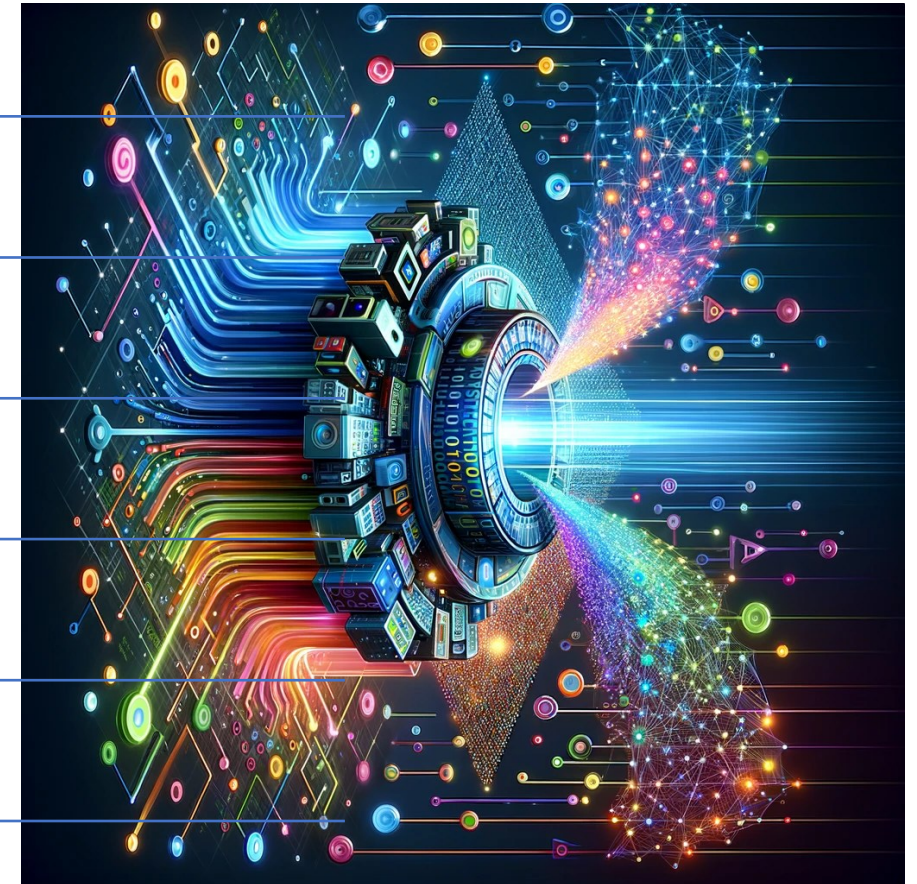
Creación de nuevas características

Manejo de datos no balanceados

Manejo de asimetría y curtosis

Manejo de categorías raras

Manejo de series de tiempo



Selección y extracción de características:

Conceptos

La selección y extracción de características implica seleccionar las características más relevantes del conjunto de datos para mejorar la precisión y eficiencia del modelo.

- **PCA.**
- **LDA**
- **Análisis de Correlación.**
- **RFE**
- **Métodos basados en los arboles**
- **Búsqueda exhaustiva**
- **Métodos integrados**



Selección y extracción de características:

PCA

```
from sklearn.decomposition import PCA
```

```
# crear un objeto PCA
```

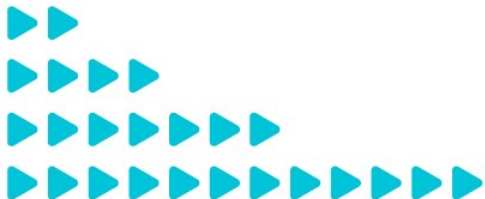
```
pca = PCA(n_components=2) o
```

```
# Preparar y transformar el dato
```

```
X_pca = pca.fit_transform(Xtrain)
```

```
# calcular la relación de varianza explicada
```

```
print("Explained variance ratio:", pca.explained_variance_ratio_)
```



Selección y extracción de características:

LDA

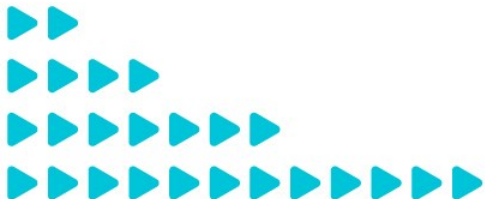
```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
```

```
# Crear un objeto LDA
```

```
LDA = LinearDiscriminantAnalysis(n_components=3)
```

```
# Preparar y transformar el dato
```

```
X_lda = LDA.fit_transform(X, y)
```



Selección y extracción de características:

Analisis de Correlación

```
import pandas as pd
```

```
# Calcular la matriz de correlacion
```

```
corr_matrix = df.corr()
```

```
# Seleccionar características altamente correlacionadas
```

```
high_corr = corr_matrix[abs(corr_matrix) > 0.8]
```

```
# Eliminar características altamente correlacionadas
```

```
df = df.drop(high_corr.columns, axis=1)
```



Selección y extracción de características:

RFE

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LinearRegression
from sklearn.datasets import load_boston

dato = load_boston()
X, y = dato.data, dato.target

modelo = LinearRegression()
rfe = RFE(modelo, n_features_to_select=5)
rfe.fit(X, y)

Características_seleccionadas= dato.feature_names[rfe.support_]
print(selected_features)
```



Selección y extracción de características:

Búsqueda exhaustiva

```
from itertools import combinations
from sklearn.metrics import r2_score
from sklearn.linear_model import LinearRegression

# Cargar el dataset
X, y = load_dataset()

# Inicializar variables
best_score = -float('inf')
best_features = None

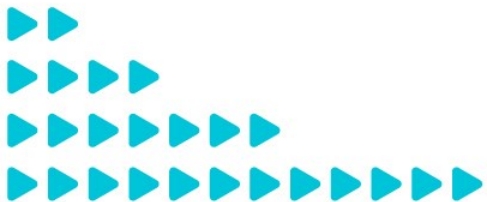
# Recorre todos los subconjuntos posibles de funciones
for k in range(1, len(X.columns) + 1):
    for subset in combinations(X.columns, k):

        # Entrenar un modelo de regresión lineal
        X_subset = X[list(subset)]
        modelo = LinearRegression().fit(X_subset, y)

        # Computar el R2
        score = r2_score(y, modelo.predict(X_subset))

        # Actualizar el mejor subconjunto de características
        if score > best_score:
            best_score = score
            best_features = subset

        # Imprimir el mejor subconjunto de características
        print(best_features)
```



Selección y extracción de características:

Regresión de Lasso

```
from sklearn.linear_model import Lasso
from sklearn.datasets import load_boston
from sklearn.preprocessing import StandardScaler
```

```
datos = load_boston()
X = datos.data
y = datos.target
```

```
# Estandarizar las características
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
# Fit el modelo Lasso
lasso = Lasso(alpha=0.1)
lasso.fit(X, y)
```

```
# Obtener los coeficientes
coefficients = lasso.coef_
```



Selección y extracción de características:

Regresión de Ridge

```
from sklearn.linear_model import Ridge
from sklearn.datasets import load_boston
from sklearn.preprocessing import StandardScaler
```

```
data = load_boston()
X = data.data
y = data.target
```

```
# Estandarizar las características
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
# Fit el modelo Ridge
ridge = Ridge(alpha=0.1)
ridge.fit(X, y)
```

```
# Obtener los coeficientes
coefficients = ridge.coef_
```

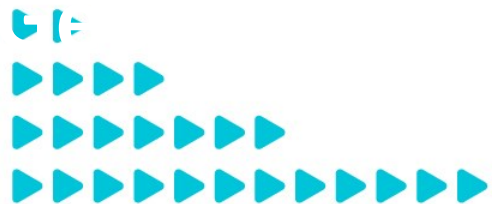


Codificación de variables categóricas

Conceptos

Codificar variables categóricas es un paso crucial en la ingeniería de características que implica convertir variables categóricas en una forma numérica que los algoritmos de aprendizaje automático puedan comprender.

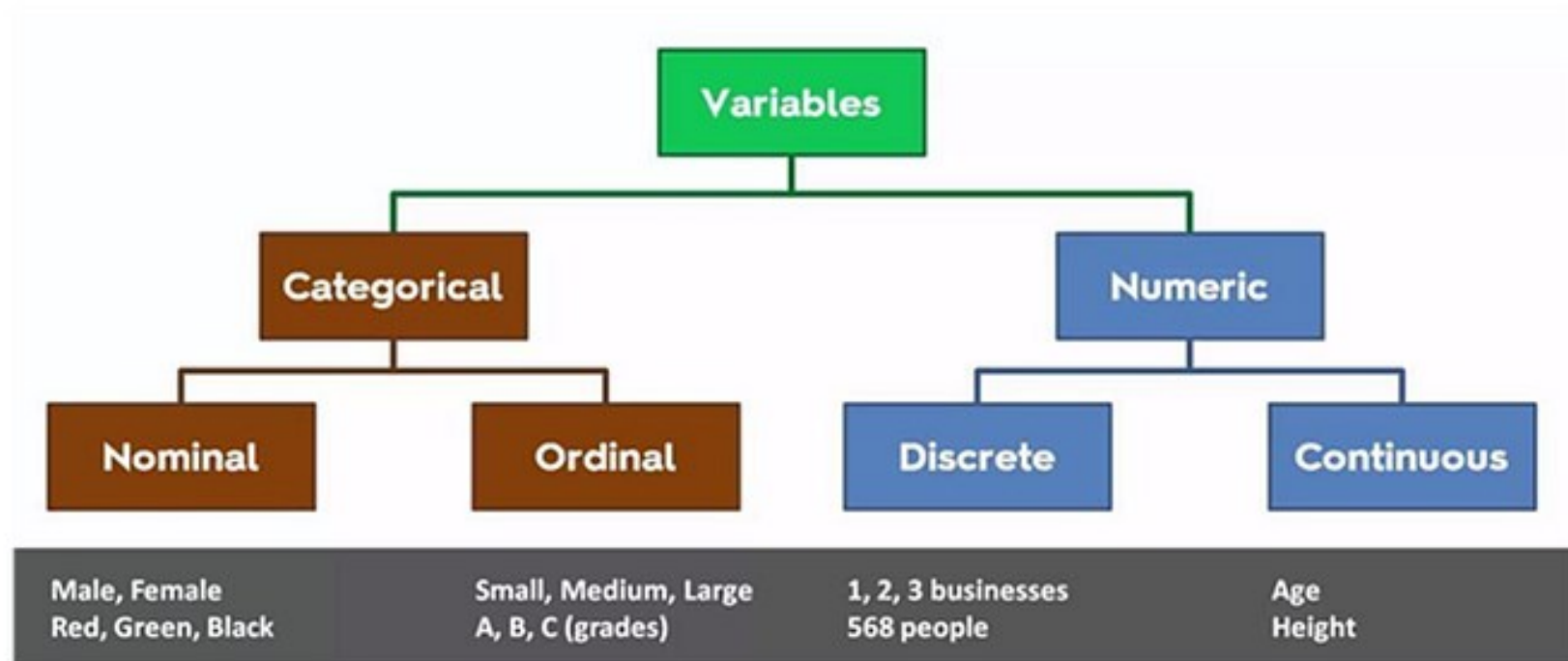
- **One-Hot Encoding**
- **Label Encoding**
- **Ordinal Encoding**



Codificación de variables categóricas

Conceptos

Codificar variables categóricas es un paso crucial en la ingeniería de características que implica convertir variables categóricas en una forma numérica que los algoritmos de aprendizaje automático puedan comprender.



One-hot Encoding

Conceptos

La **codificación one-hot** es una técnica que convierte variables categóricas en un conjunto de características binarias, donde cada característica corresponde a una categoría única en la variable original. En esta técnica, se crea una nueva columna binaria para cada categoría y el valor se establece en 1 si la categoría está presente y 0 si no.

```
import pandas as pd

# Crear un dataframe
df = pd.DataFrame({'color': ['red', 'blue', 'green', 'red',
                              'yellow', 'blue']})

# aplicar one-hot encoding
one_hot_encoded = pd.get_dummies(df['color'])
print(one_hot_encoded)
```



Label Encoding

Conceptos

La **codificación de etiquetas** es una técnica que asigna un valor numérico único a cada categoría en la variable original. En esta técnica, a cada categoría se le asigna una etiqueta numérica, donde las etiquetas se asignan según el orden de las categorías en la variable.

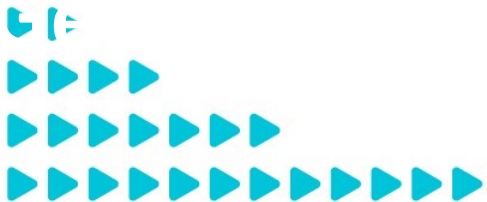
```
from sklearn.preprocessing import LabelEncoder
```

```
# Crear las muestras del dataframe
```

```
df = pd.DataFrame({  
    'color': ['red', 'blue', 'green', 'red', 'yellow', 'blue']  
})
```

```
# Aplicar label encoding
```

```
label_encoder = LabelEncoder()  
df['color_encoded'] =  
    label_encoder.fit_transform(df['color'])  
print(df)
```



Ordinal Encoding

Conceptos

La **codificación ordinal** es una técnica que asigna un valor numérico a cada categoría en la variable original en función de su orden o rango. En esta técnica, las categorías se ordenan en función de un criterio específico y a las categorías se les asignan valores numéricos en función de su posición en el orden.

```
import category_encoders as ce
```

```
# Crear muestras de dataframe
```

```
df = pd.DataFrame({  
    'size': ['S', 'M', 'L', 'XL', 'M', 'S']  
})
```

```
# Aplicar ordinal encoding
```

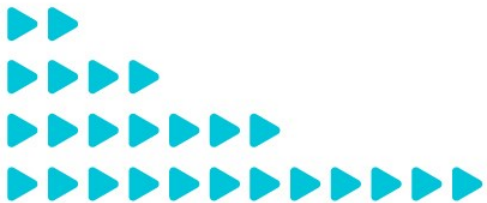
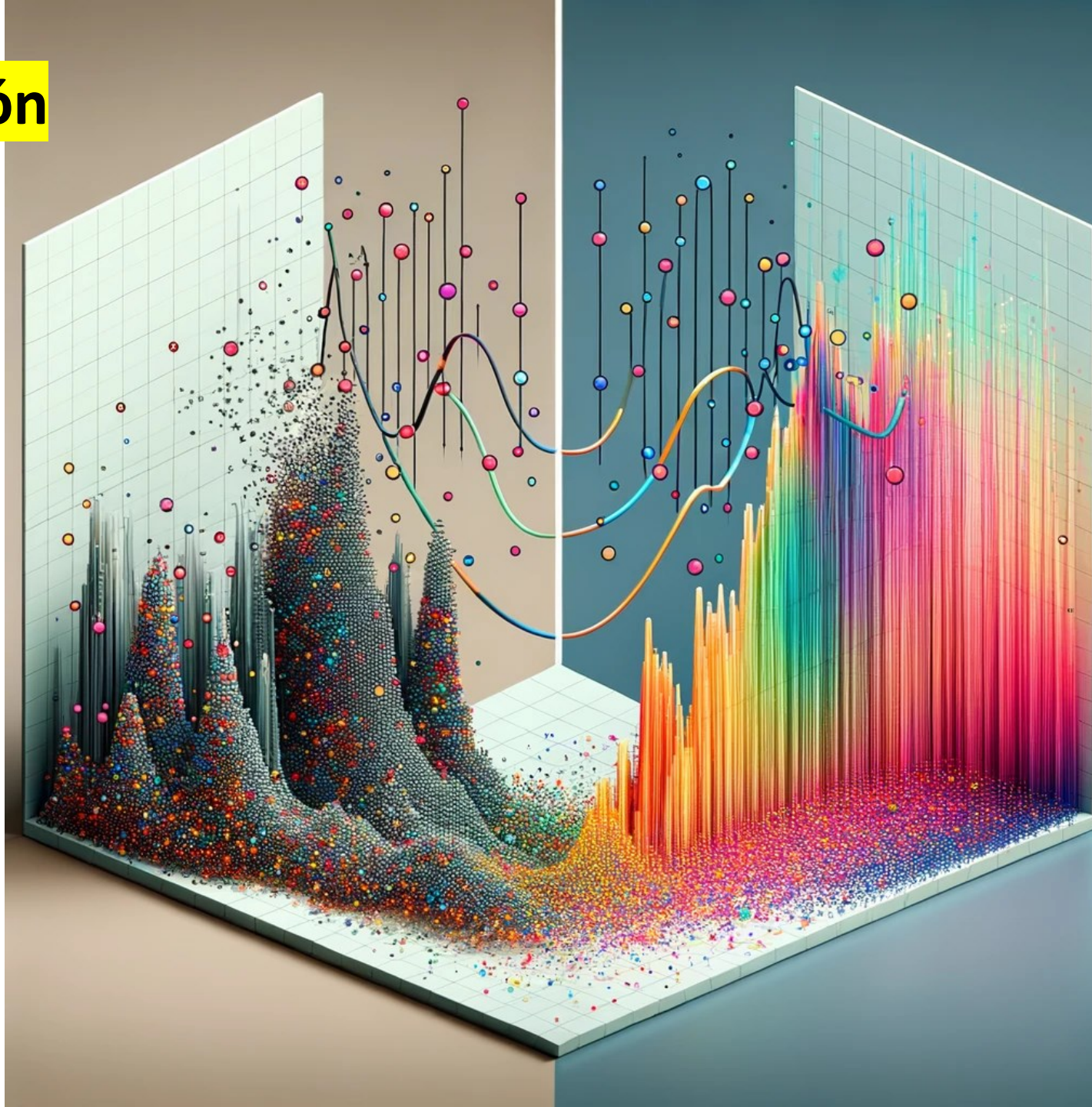
```
ordinal_encoder = ce.OrdinalEncoder(cols=['size'],  
    order=['S', 'M', 'L', 'XL'])  
df = ordinal_encoder.fit_transform(df)  
print(df)
```



Escalamiento y normalización

Conceptos

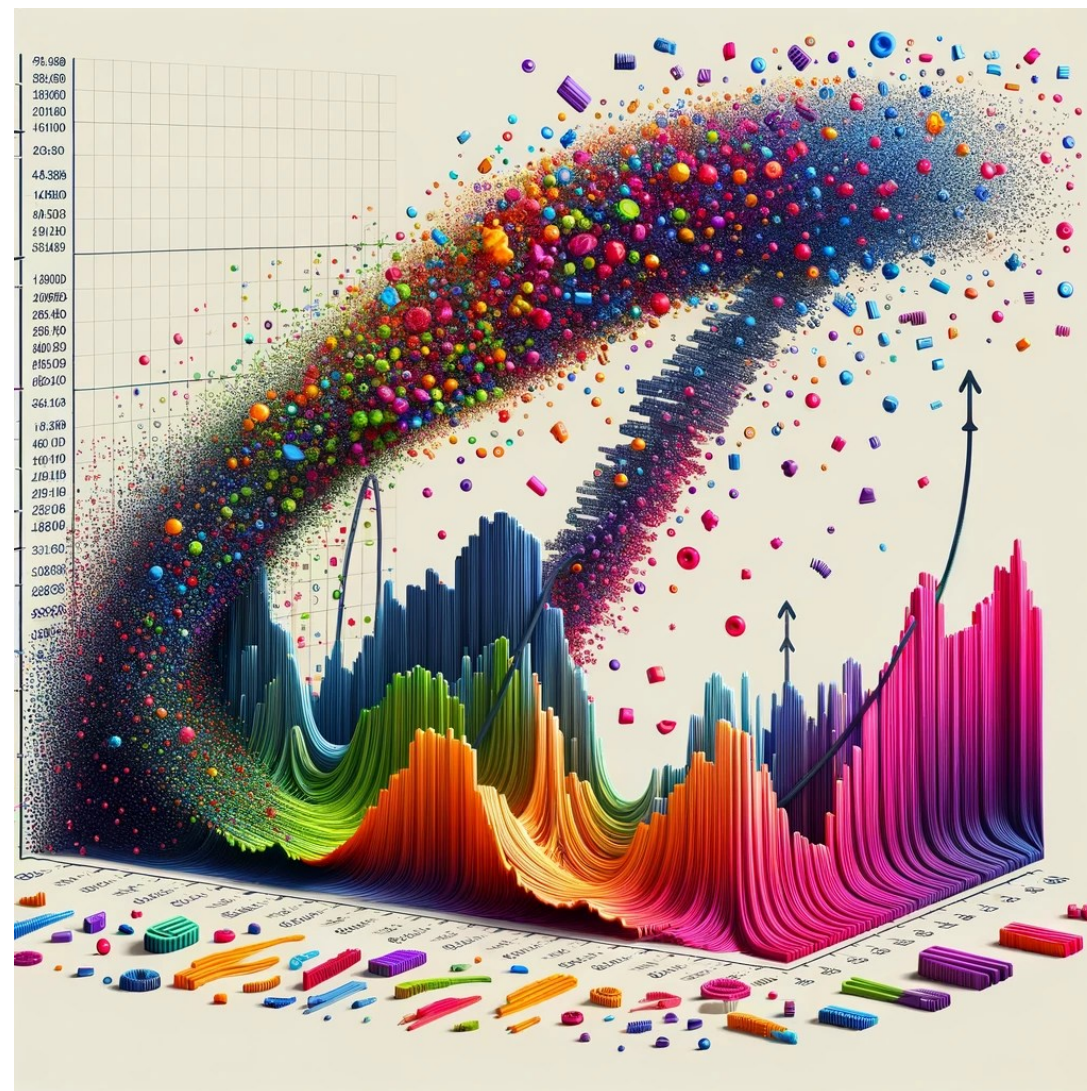
El escalado y la normalización son pasos importantes en la ingeniería de funciones para garantizar que las funciones estén en una escala similar y tengan rangos similares. Esto puede ayudar a mejorar el rendimiento de algunos algoritmos de aprendizaje automático y acelerar el proceso de optimización.



Preparación de datos

Conceptos

- Estandarización
- Escalamiento Min-Max
- Escalamiento robusto
- Normalización

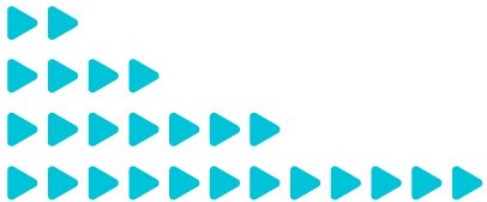
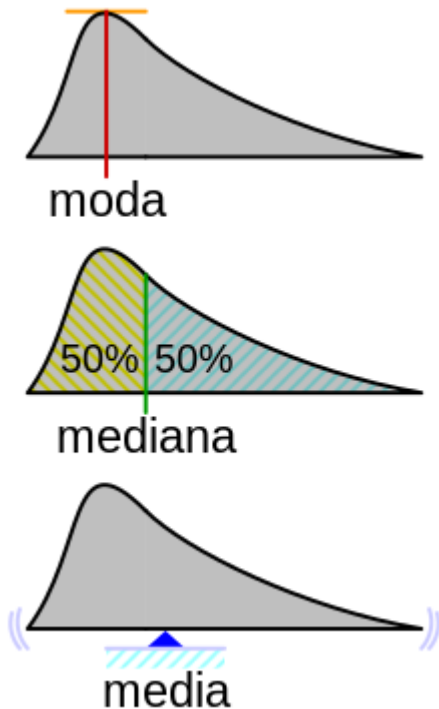


Estandarización

Conceptos

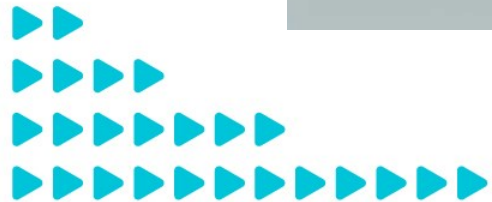
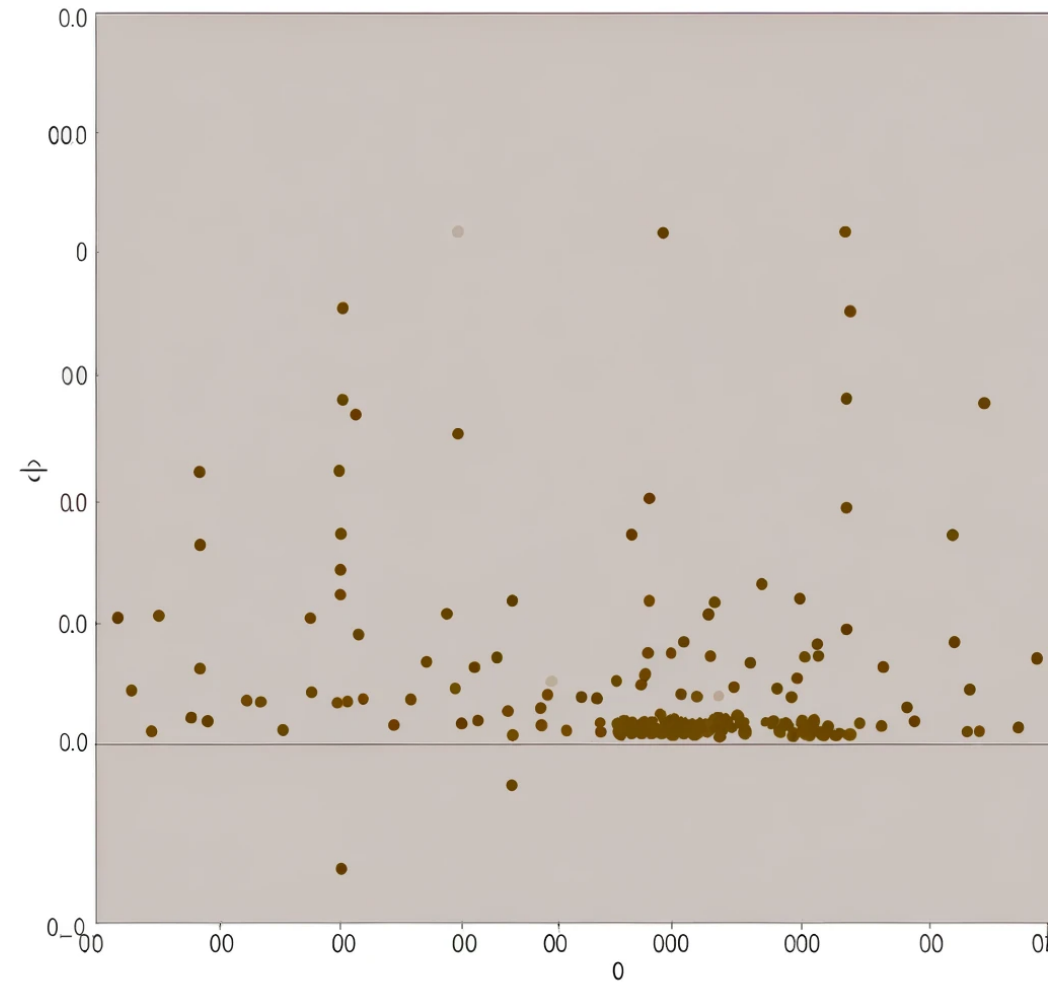
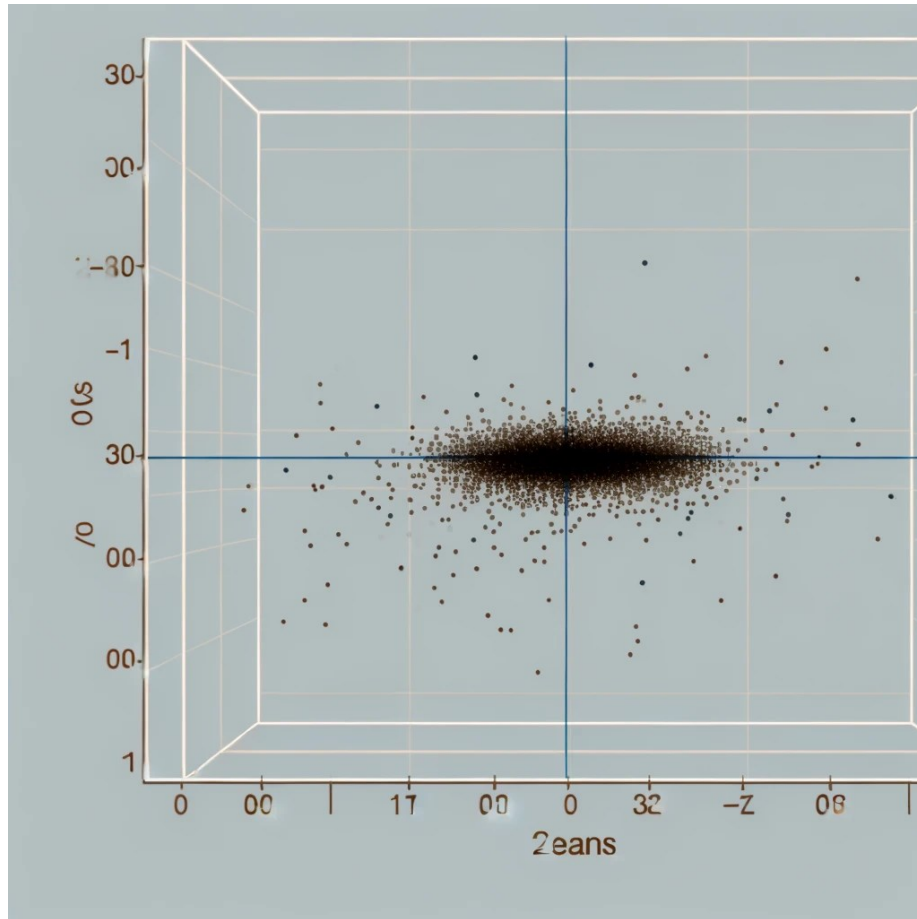
La estandarización escala las características para que tengan media cero y varianza unitaria. Esto se hace restando la media de cada valor y luego dividiéndola por la desviación estándar. Los valores resultantes tendrán una media de cero y una desviación estándar de uno.

- Comparabilidad
- Mejora del rendimiento de los algoritmos
- Convergencia más rápida
- Interpretación más fácil



Estandarización

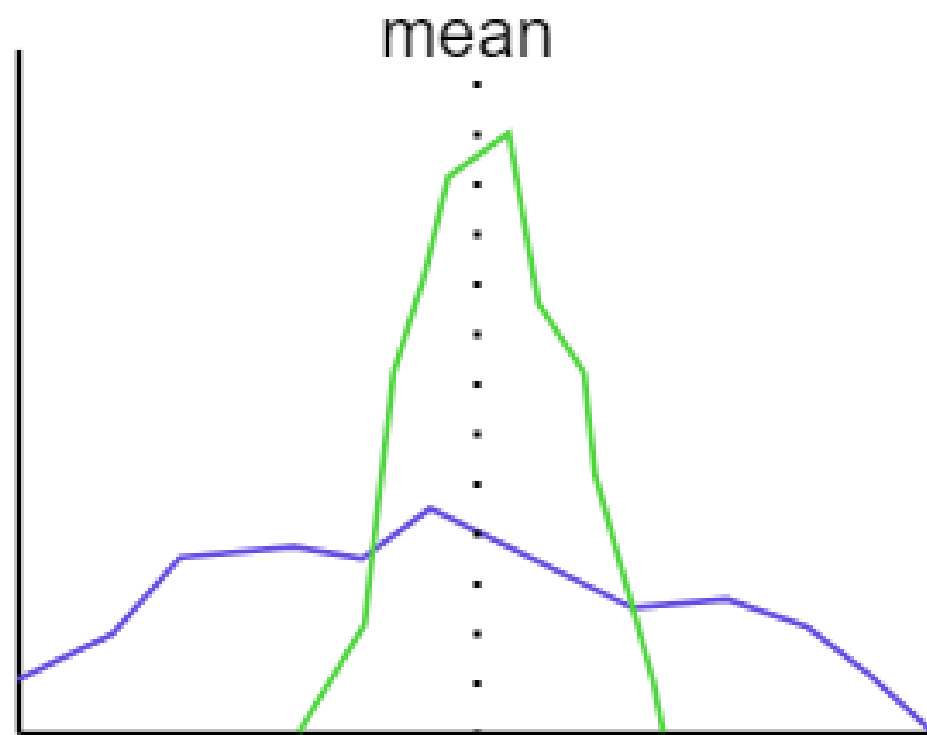
Media = 0



Estandarización

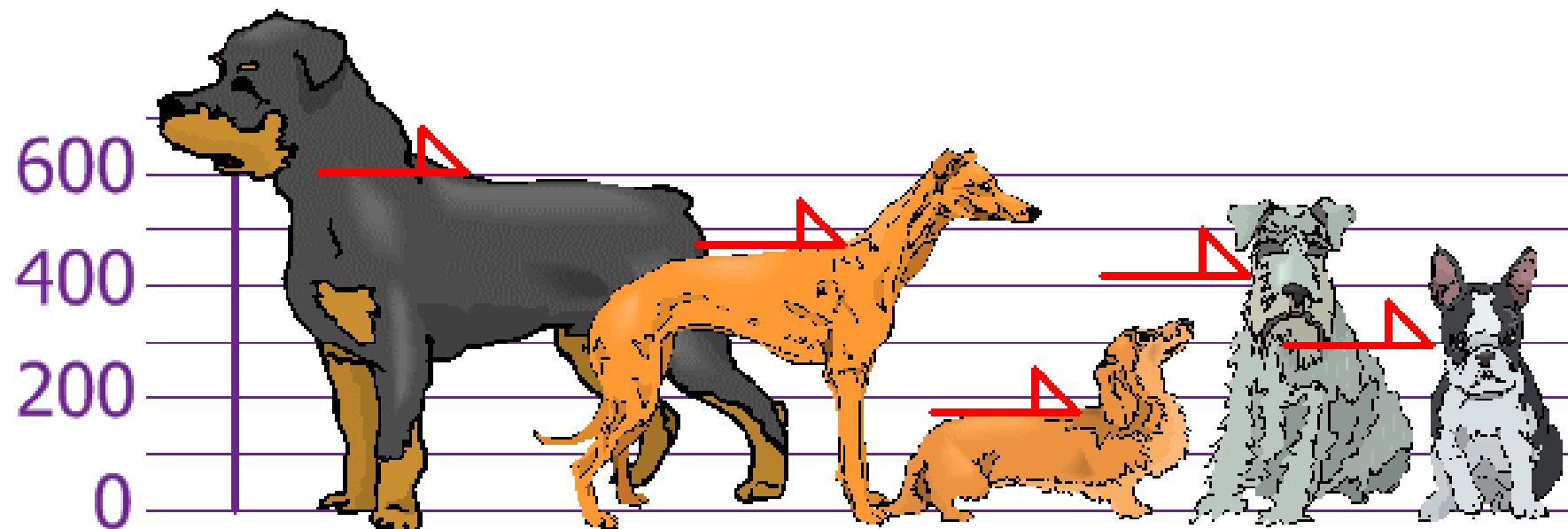
Varianza unitaria

Significa que después de ajustar los datos, la varianza (que mide la dispersión de los datos respecto a la media) se escala para ser 1. Esto se logra dividiendo cada dato por la desviación estándar original del conjunto de datos. Matemáticamente, después de restar la media y dividir cada dato por la desviación estándar original, la varianza del nuevo conjunto de datos ($\text{Var}(X')$) será 1. Tener varianza unitaria estandariza la escala de los datos, lo que permite que características con diferentes unidades o escalas de medida sean comparables y contribuyan por igual al análisis.



Estandarización

Recordemos



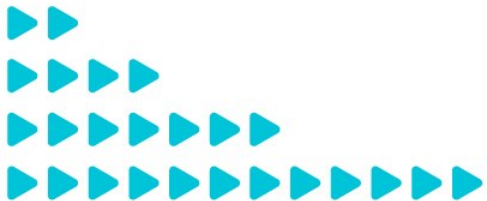
Estandarización

Conceptos

```
from sklearn.preprocessing import StandardScaler
```

```
# Crear un objeto StandardScaler  
scaler = StandardScaler()
```

```
# Preparar y transformer los datos  
X_scaled = scaler.fit_transform(X)
```



Estandarización de Min - Max

Conceptos

La Estandarización Min-Max escala las características a un rango fijo, generalmente entre 0 y 1. Esto se hace restando el valor mínimo de cada valor y luego dividiendo por el rango.

```
from sklearn.preprocessing import MinMaxScaler
```

```
# Crear un objeto MinMaxScaler  
scaler = MinMaxScaler()
```

```
# Preparar y transformar el dato  
X_scaled = scaler.fit_transform(X)
```



Estandarización Robusta

Conceptos

La estandarización robusta es similar a la estandarización, pero utiliza la mediana y el rango intercuartílico en lugar de la media y la desviación estándar. Esto lo hace más robusto ante los valores atípicos de los datos.

```
from sklearn.preprocessing import RobustScaler
```

```
# Crear un objeto Robusto
```

```
scaler = RobustScaler()
```

```
# Preparar y transformar el dato
```

```
X_scaled = scaler.fit_transform(X)
```



Normalización

Conceptos

La normalización escala cada observación para que tenga una norma unitaria, lo que significa que la suma de los cuadrados de cada valor de característica es 1. Esto es útil para algunos algoritmos que requieren una escala similar para todas las muestras.

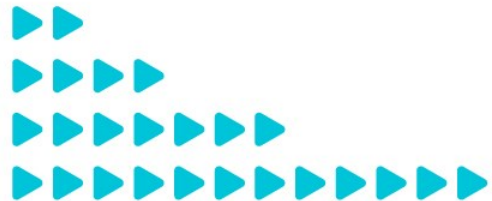
```
from sklearn.preprocessing import Normalizer()
```

```
# Crear un objeto Normalizado
```

```
scaler = Normalizer()
```

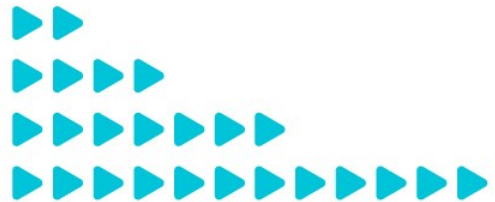
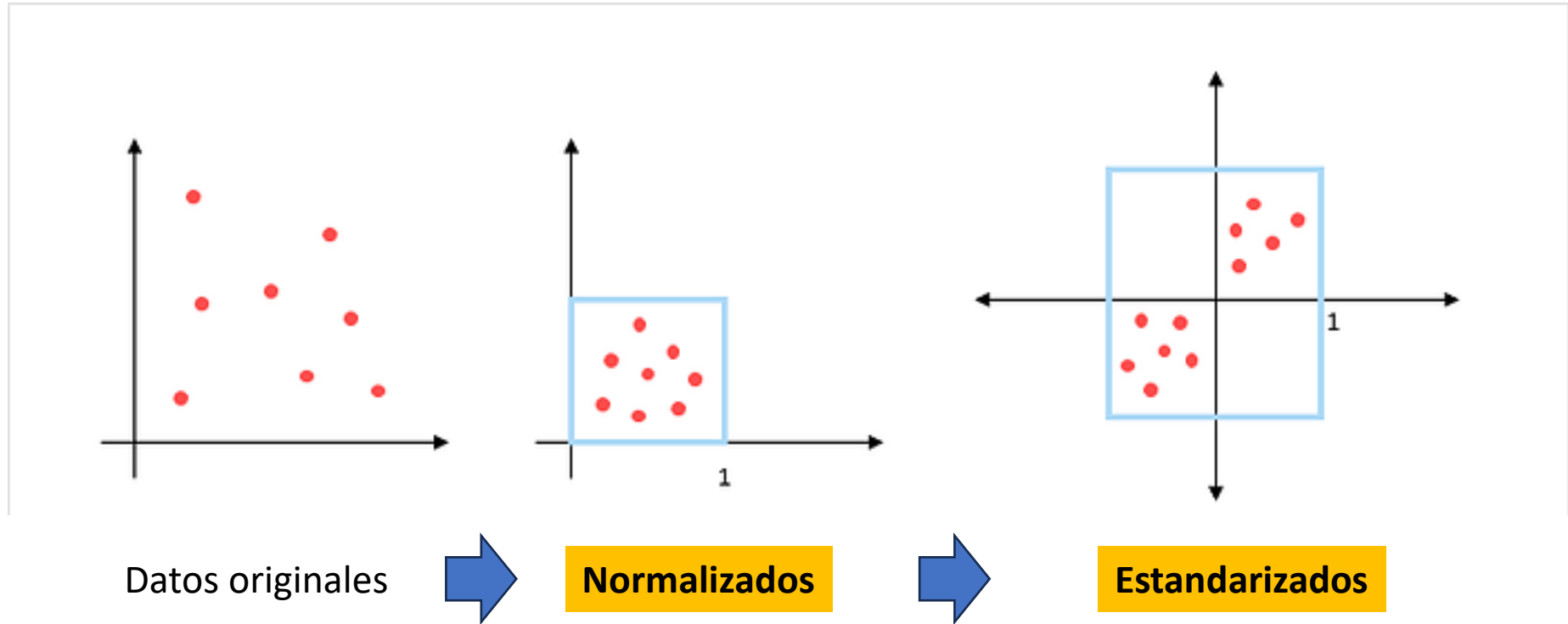
```
# Preparar y transformar el dato
```

```
X_normalizado = scaler.fit_transform(X)
```



Escalado y Normalización

Conceptos



Características de interacción

Conceptos

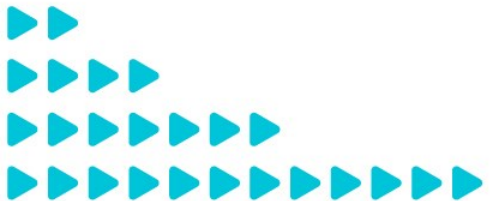
Se crean multiplicando dos o más funciones existentes. Esto puede ayudar a capturar los efectos conjuntos de las características y descubrir nuevos patrones en los datos. Por ejemplo, si tenemos dos características, "edad" e "ingresos", podemos crear una nueva característica de interacción llamada "edad_ingresos" multiplicando estas dos características.

```
import pandas as pd

# Creemos data frame
dato = pd.DataFrame({'age': [25, 30, 35],
                    'income': [50000, 60000, 70000]})

# Crear una nueva característica interactiva
dato['age_income'] = dato['age'] * dato['income']

# Graficar el dataframe actualizado
print(dato)
```



Características Polinomiales

Conceptos

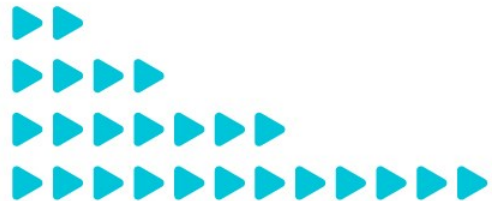
Se crean elevando las características existentes a una potencia mayor. Esto puede ayudar a capturar relaciones no lineales entre las características y mejorar la precisión de los modelos. Por ejemplo, si tenemos una característica "edad", podemos crear una nueva característica polinómica llamada "age_squared" elevando esta característica al cuadrado.

```
from sklearn.preprocessing import PolynomialFeatures
import numpy as np
```

```
# crear un dataset
X = np.array([[1, 2],
              [3, 4]])
```

```
# crear una característica polinomial
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)
```

```
# Graficar la característica actualizada
print(X_poly)
```



Binning

Conceptos

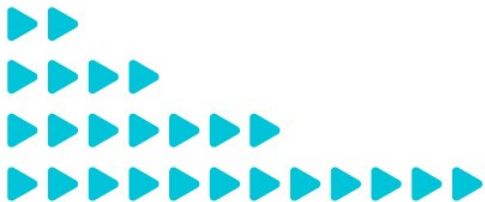
Binning implica agrupar valores continuos en categorías discretas. Esto puede ayudar a capturar relaciones no lineales y reducir el impacto de los valores atípicos en los datos. Por ejemplo, si tenemos una función "edad", podemos crear una nueva función agrupada llamada "grupo_edad" agrupando las edades en diferentes categorías, como "0-18", "18-25", "25-35", "35-50" y "50+".

```
import pandas as pd

# crear un data frame
dato = pd.DataFrame({'age': [20, 25, 30, 35, 40, 45, 50, 55]})

# crear bins para diferentes grupos de edades
bins = [0, 18, 25, 35, 50, float('inf')]
labels = ['0-18', '18-25', '25-35', '35-50', '50+']
dato['age_group'] = pd.cut(dato['age'], bins=bins, labels=labels)

# Graficar el dataframe actualizado
print(data)
```

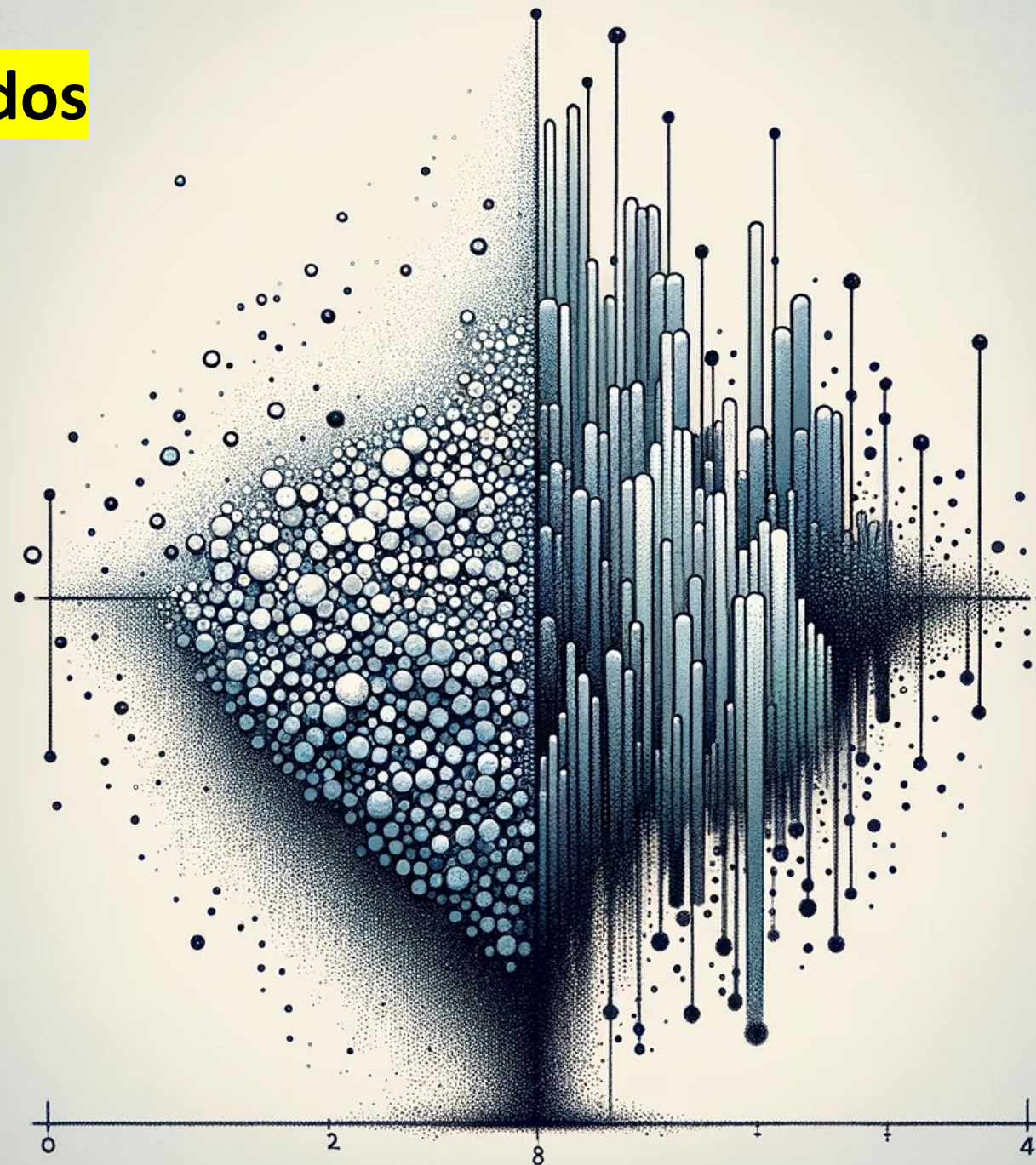


Manejo de datos desbalanceados

Conceptos

Los datos desequilibrados son una situación en la que la distribución de la variable objetivo no es uniforme y una clase está subrepresentada en comparación con la otra.

Esto puede llevar a un sesgo en el modelo hacia la clase mayoritaria, y el modelo puede tener un mal desempeño en la clase minoritaria.



Upsampling

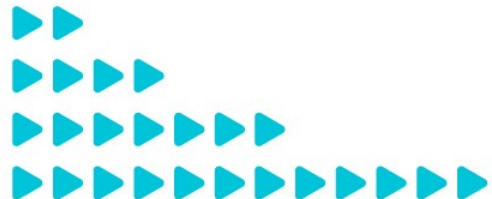
Conceptos

El **Upsamplig** o **sobremuestreo** implica crear más muestras para la clase minoritaria volviendo a muestrear las muestras existentes con reemplazo. Esto se puede hacer usando la función **resample** del módulo **sklearn.utils**

```
from sklearn.utils import resample
```

```
# Upsample clase minoritaria
```

```
X_upsampled, y_upsampled = resample(X_minority,  
y_minority, replace=True, n_samples=len(X_majority),  
random_state=42)
```



Downsampling

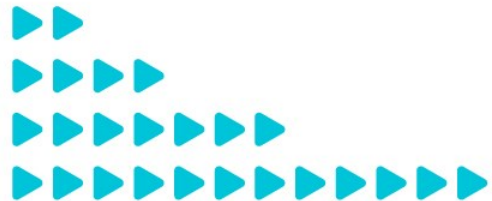
Conceptos

La reducción de resolución implica eliminar algunas muestras de la clase mayoritaria para equilibrar la distribución. Esto se puede hacer usando la función **resample** del módulo **sklearn.utils**.

```
from sklearn.utils import resample
```

```
# Downsample majority class
```

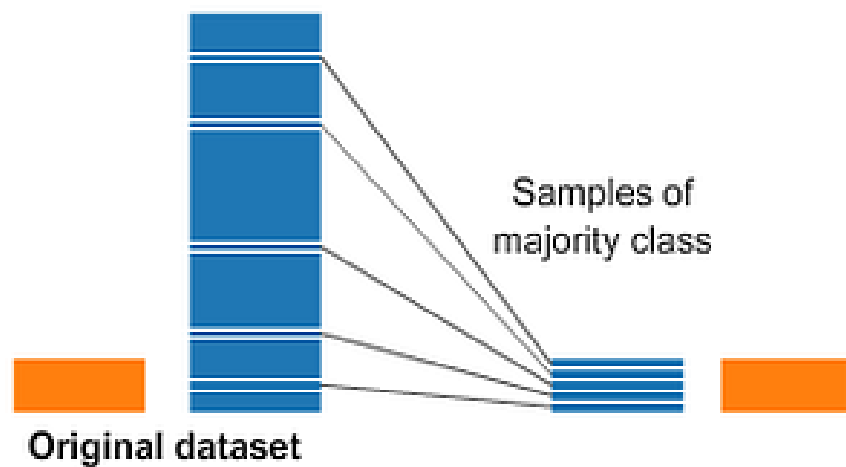
```
X_downsampled, y_downsampled = resample(X_majority,  
y_majority, replace=False, n_samples=len(X_minority),  
random_state=42)
```



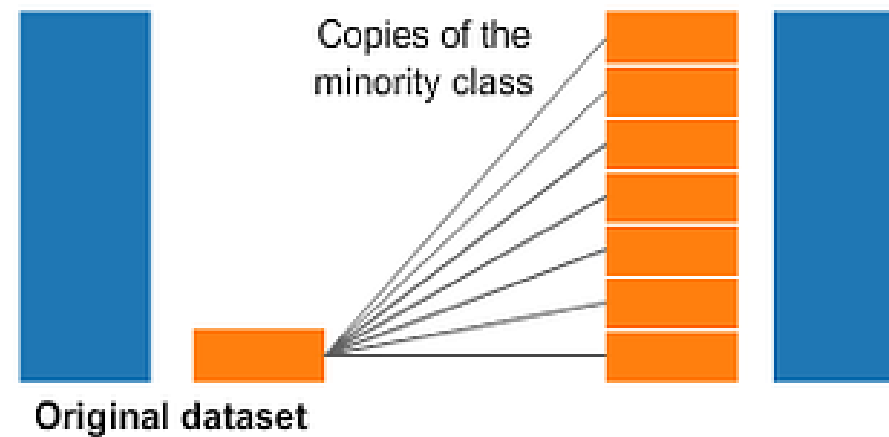
Downsampling & Upsampling

Conceptos

Undersampling



Oversampling



Técnica de sobremuestreo de minorías sintéticas (SMOTE)

Conceptos

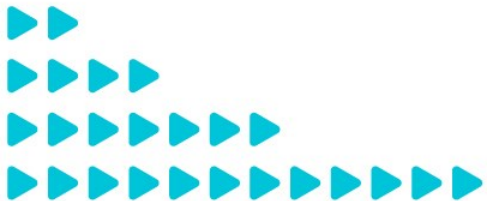
SMOTE implica la creación de muestras sintéticas para la clase minoritaria basándose en las muestras existentes. Esto se puede hacer usando la función **SMOTE** del módulo **imblearn.over_sampling**.

```
from imblearn.over_sampling import SMOTE
```

```
# Usar SMOTE para generar mas datos de la clase minoritaria
```

```
sm = SMOTE(random_state=42)
```

```
X_resampled, y_resampled = sm.fit_resample(X, y)
```



Ponderación de clase

Conceptos

Implica asignar un peso a cada clase en el modelo para tener en cuenta el desequilibrio. Esto se puede hacer usando el parámetro **class_weight** en el modelo

```
from sklearn.linear_model import LogisticRegression
```

```
# Usar class weighting para manejar el imbalance
```

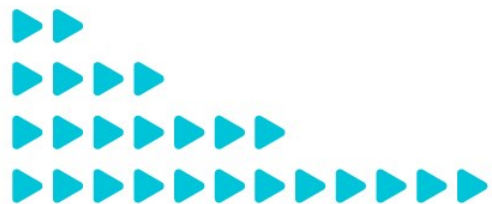
```
clf = LogisticRegression(class_weight='balanced', random_state=42)  
clf.fit(X_train, y_train)
```



Anomalías en datos desbalanceados

Conceptos

En el caso de datos desequilibrados, donde el número de observaciones en una clase es mucho menor que en la otra, la detección de anomalías se puede utilizar para identificar y etiquetar las observaciones raras en la clase minoritaria como anomalías. Esto puede ayudar a equilibrar el conjunto de datos y mejorar el rendimiento de los modelos de aprendizaje automático.



Anomalías en datos desbalanceados

Conceptos

Un enfoque común para la detección de anomalías en datos desequilibrados es utilizar técnicas de aprendizaje no supervisadas, como la agrupación, donde las observaciones de clases minoritarias se agrupan en grupos distintos según sus similitudes.

Las observaciones en la clase minoritaria que no pertenecen a ninguno de estos grupos pueden etiquetarse como anomalías.



Anomalías en datos desbalanceados

Conceptos

Otro enfoque es utilizar técnicas de aprendizaje supervisado, como la clasificación de una clase, donde se entrena un modelo con los datos de la clase mayoritaria para aprender el comportamiento normal de los datos. Las observaciones de la clase minoritaria que se desvían significativamente del comportamiento normal aprendido se etiquetan como anomalías.

```
from sklearn.ensemble import IsolationForest
```

```
# Use anomaly detection to handle imbalance
```

```
clf = IsolationForest(random_state=42)
```

```
clf.fit(X_train)
```

```
X_train = X_train[clf.predict(X_train) == 1]
```

```
y_train = y_train[clf.predict(X_train) == 1]
```



Aprendizaje sensible a los costos

Conceptos

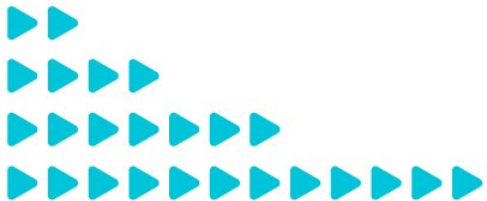
Implica asignar un costo diferente a cada tipo de error en el modelo para tener en cuenta el desequilibrio. Esto se puede hacer usando **sample_weight** en el modelo.

```
from sklearn.tree import DecisionTreeClassifier
```

```
# Usar cost-sensitive learning para manejar el imbalance
```

```
clf = DecisionTreeClassifier(random_state=42)
```

```
clf.fit(X_train, y_train, sample_weight=class_weights)
```



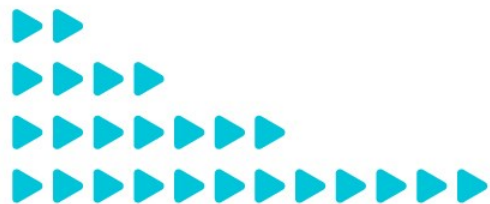
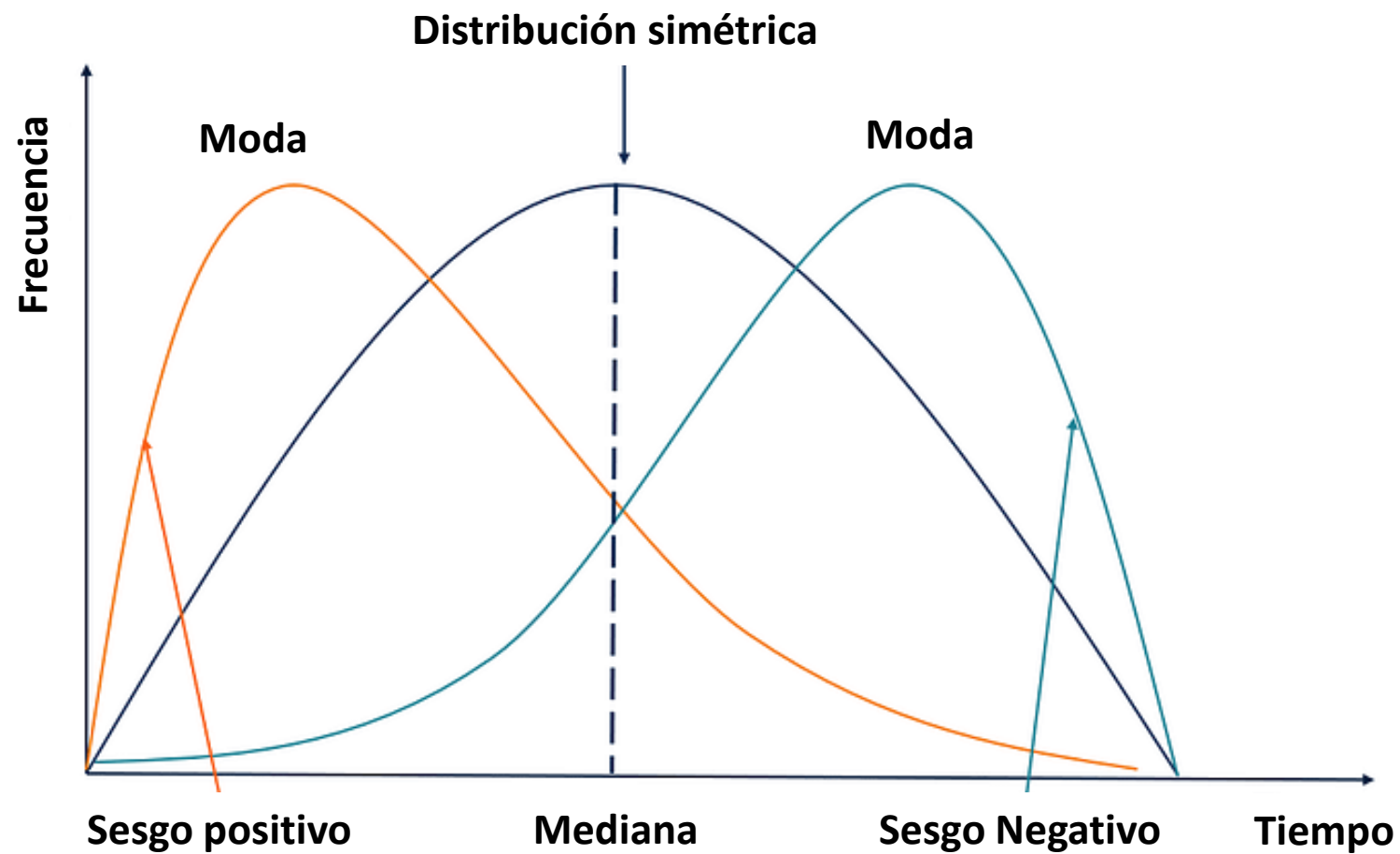
EDA

Manejo de asimetría y curtosis



Simetría

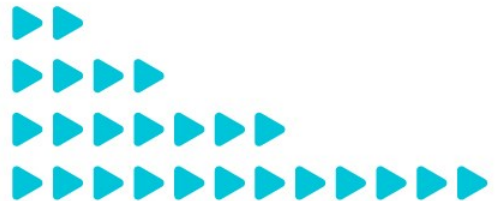
Conceptos



Técnicas para manejar la asimetría

Conceptos

- Log transformation
- Square root transformation
- Box-Cox transformation



Técnicas para manejar la asimetría

Conceptos

```
import numpy as np
import pandas as pd
from scipy import stats

# Generar algún dato asimetrico
dato = np.random.gamma(1, 10, 1000)

# Calcular asimetría y kurtosis
skewness = stats.skew(dato)
kurtosis = stats.kurtosis(dato)

print("Skewness:", skewness)
print("Kurtosis:", kurtosis)

# Log transformation
log_data = np.log(dato)
log_skewness = stats.skew(log_data)
log_kurtosis = stats.kurtosis(log_data)
```



Técnicas para manejar la asimetría

Conceptos

```
print("Log Skewness:", log_skewness)
print("Log Kurtosis:", log_kurtosis)
```

Transformación de la raíz cuadrada

```
sqrt_data = np.sqrt(data)
sqrt_skewness = stats.skew(sqrt_data)
sqrt_kurtosis = stats.kurtosis(sqrt_data)
```

```
print("Sqrt Skewness:", sqrt_skewness)
print("Sqrt Kurtosis:", sqrt_kurtosis)
```

Box-Cox transformation

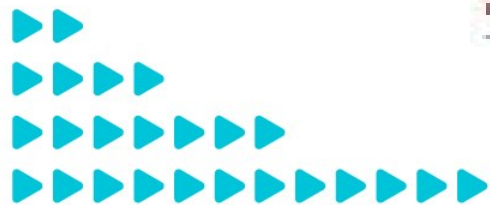
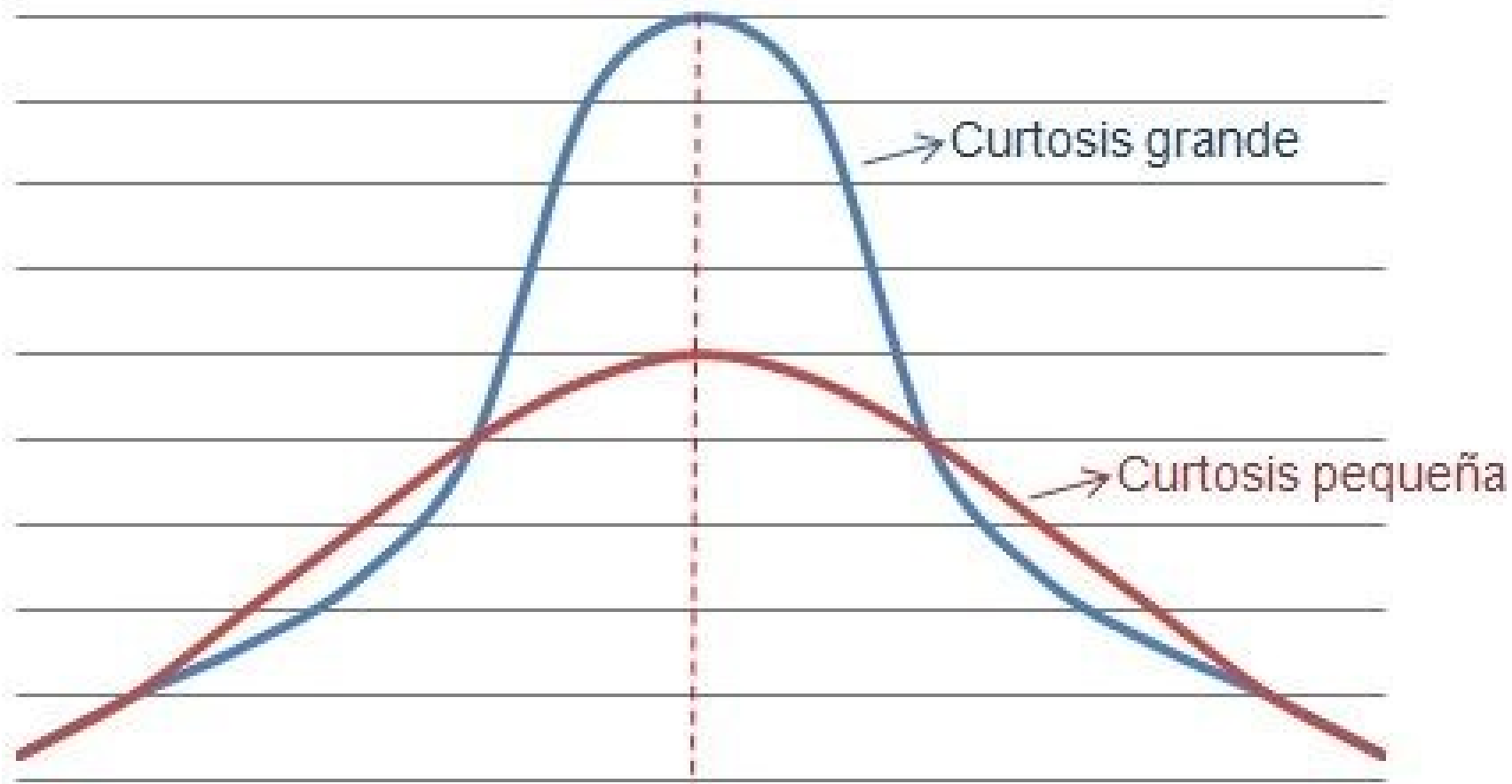
```
box_cox_data, _ = stats.boxcox(data)
box_cox_skewness = stats.skew(box_cox_data)
box_cox_kurtosis = stats.kurtosis(box_cox_data)
```

```
print("Box-Cox Skewness:", box_cox_skewness)
print("Box-Cox Kurtosis:", box_cox_kurtosis)
```



Curtosis

Conceptos

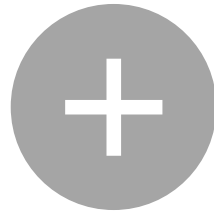


Curtosis

Clases



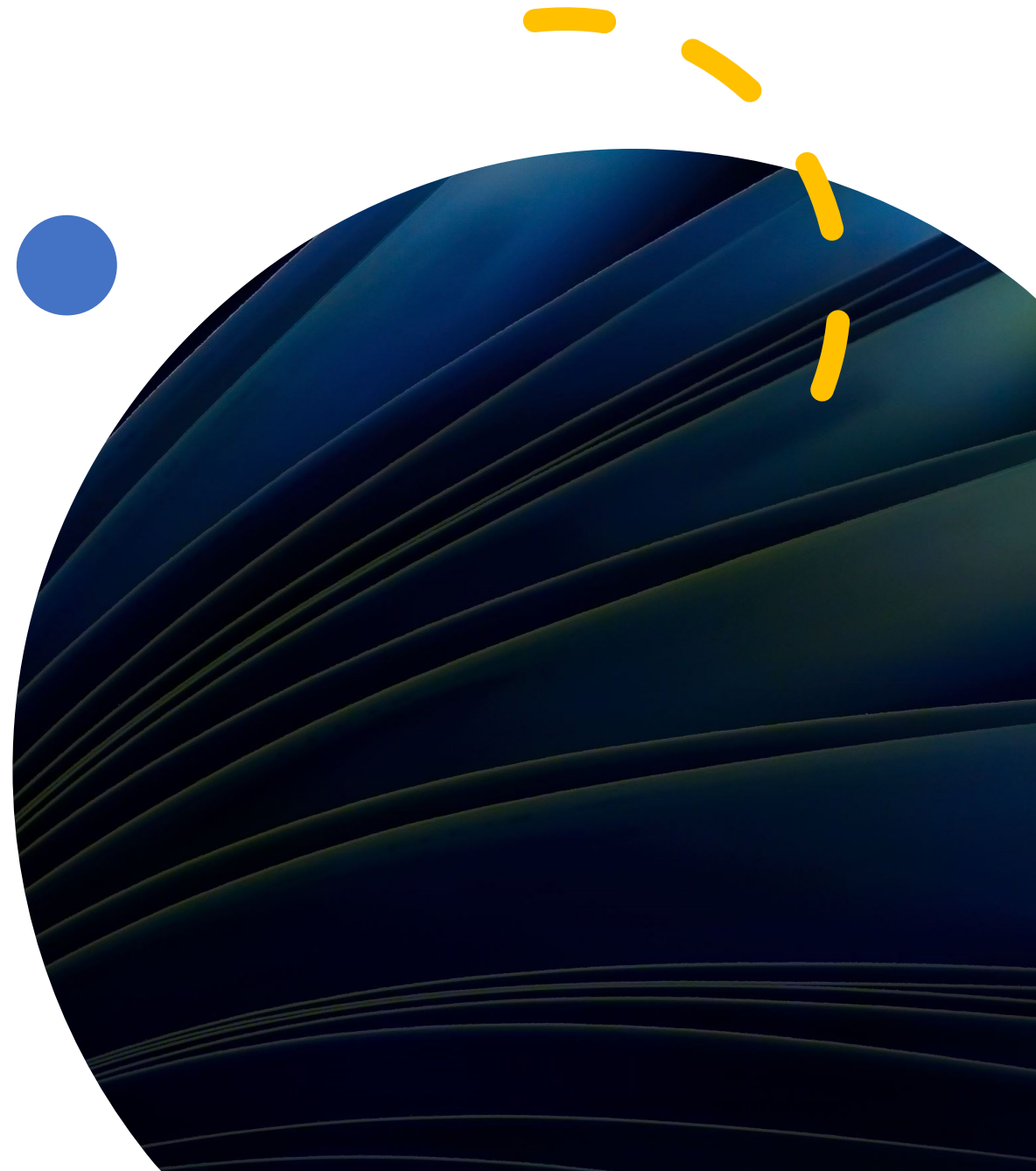
LEPTOCÚRTICA:
LA CURTOSIS > 0 . LOS DATOS
ESTÁN MUY
CONCENTRADOS EN LA
MEDIA, SIENDO UNA
CURVA MUY APUNTADA.



MESOCÚRTICA:
LA CURTOSIS $= 0$.
DISTRIBUCIÓN NORMAL.



PLATICÚRTICA:
LA CURTOSIS < 0 . MUY POCA
CONCENTRACIÓN DE
DATOS EN LA MEDIA,
PRESENTANDO UNA
FORMA MUY ACHATADA.



Curtosis

Implicaciones de una Curtosis Alta en Datos de Distribución para el Entrenamiento Supervisado

Mayor Sensibilidad a Outliers:

Una curtosis alta indica colas más pesadas en la distribución de los datos, lo que significa que hay una mayor probabilidad de observar valores extremos (outliers).

Esto puede afectar el entrenamiento de modelos supervisados al hacerlos más sensibles a estos outliers, potencialmente llevando a un sobreajuste si el modelo aprende de estos puntos extremos como si fueran representativos del conjunto de datos general.

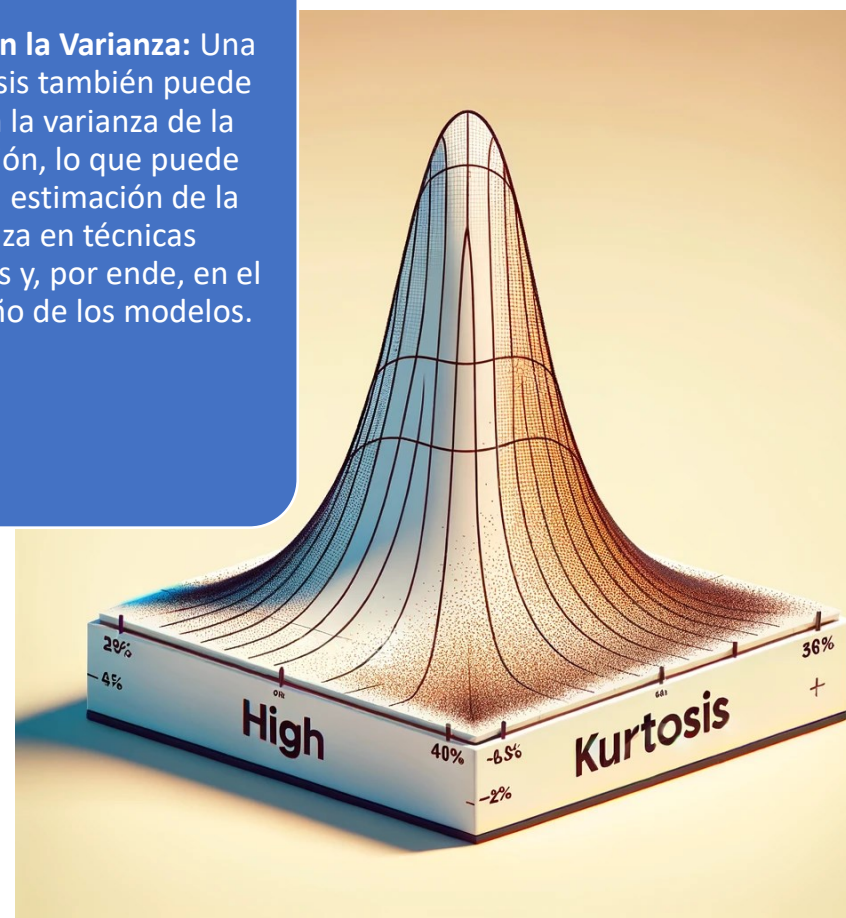


Necesidad de

Preprocesamiento: Para manejar la sensibilidad a outliers y asegurar que el modelo generalice bien, puede ser necesario aplicar técnicas de preprocesamiento de datos como la eliminación de outliers, la transformación de datos para reducir la curtosis, o el uso de modelos robustos a outliers.



Impacto en la Varianza: Una alta curtosis también puede influir en la varianza de la distribución, lo que puede afectar la estimación de la varianza en técnicas estadísticas y, por ende, en el desempeño de los modelos.



Curtosis

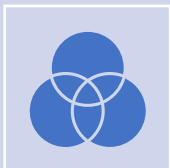
Implicaciones de una Curtosis Baja en Datos de Distribución para el Entrenamiento Supervisado



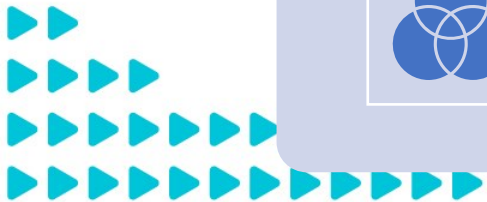
Menor Probabilidad de Valores Extremos: Una curtosis baja indica una distribución más aplanada con colas más ligeras, lo que significa que hay menos probabilidad de observar valores extremos. Esto puede hacer que los modelos sean menos susceptibles a outliers, pero también puede indicar que la distribución de los datos no captura bien la variabilidad en los datos reales.



Distribución Potencialmente Más Uniforme: Una baja curtosis puede implicar que los datos están más uniformemente distribuidos, lo que podría facilitar el entrenamiento de modelos al reducir el impacto de los outliers. Sin embargo, también puede significar que el modelo puede no ser tan efectivo para identificar patrones sutiles en los datos.



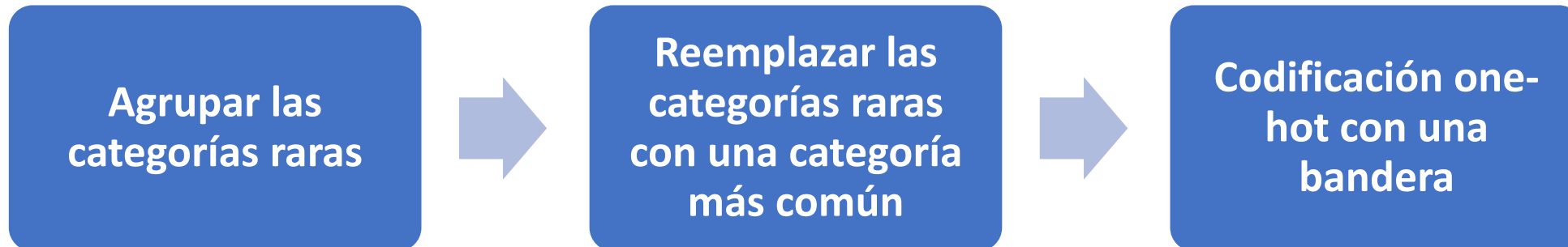
Consideraciones de Modelado: Con una curtosis baja, puede ser menos crítico aplicar técnicas de preprocesamiento intensivas para manejar outliers, aunque siempre es importante evaluar la distribución de los datos y su impacto en el modelo específico que se está entrenando.



Manejo de categorías raras

Clases

Se refiere al proceso de manejar categorías en variables categóricas que ocurren con poca frecuencia en los datos. Las categorías raras pueden causar problemas en los modelos de aprendizaje automático, ya que es posible que no tengan suficiente representación en los datos para modelarse con precisión.



Manejo de categorías raras

Clases

```
# Cargar Titanic dataset
```

```
titanic = pd.read_csv('Titanic_Data.csv')
```

```
# Ver los valores en la columna 'Embarked'
```

```
print(titanic['Embarked'].value_counts())
```

```
# Agrupar categorías raras en una categoría simple
```

```
titanic['Embarked'] = np.where(titanic['Embarked'].isin(['C', 'Q']), titanic['Embarked'], 'R')
```

```
# Ver los valores de 'Embarked' después del agrupamiento
```

```
print(titanic['Embarked'].value_counts())
```

```
# Reemplazar las categorías raras
```

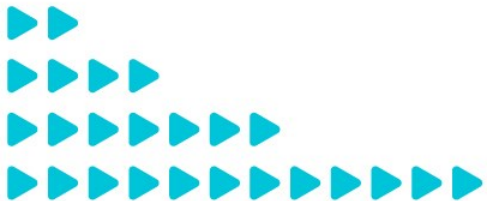
```
titanic['Embarked'] = np.where(titanic['Embarked'].isin(['C', 'Q']), titanic['Embarked'], 'S')
```

```
# Ver los valores de 'Embarked' después del reemplazamiento
```

```
print(titanic['Embarked'].value_counts())
```

```
# Crear una nueva categoría para las categorías raras
```

```
titanic['Embarked_R'] = np.where(titanic['Embarked'].isin(['C', 'Q']), 0, 1)
```



Pipeline en Machine learning

Clases



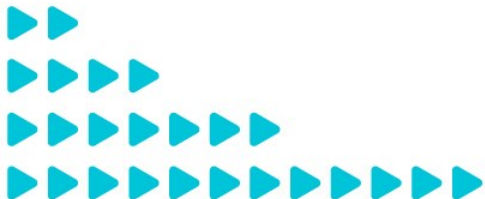
El uso de **Pipeline** en **scikit-learn** es una práctica común en casi todos los tipos de proyectos de machine learning, desde la clasificación y la regresión hasta el clustering y la reducción de dimensionalidad.



La utilidad y versatilidad de **Pipeline** radican en su capacidad para encadenar secuencialmente pasos de preprocesamiento y modelos, asegurando que todas las transformaciones se apliquen de manera coherente durante tanto el entrenamiento como las predicciones.



Inspiración Unix: La inspiración para Pipeline proviene de la filosofía Unix de "hacer una cosa bien", permitiendo que herramientas pequeñas se encadenen para realizar tareas complejas.



Pipeline en Machine learning

Cuando usarlo

1. Automatización del Flujo de Trabajo: En situaciones donde tienes varios pasos de preprocesamiento (como imputación de valores faltantes, normalización/estandarización, codificación de variables categóricas) seguidos por un modelo de machine learning. Pipeline automatiza este flujo, haciendo el código más organizado y reduciendo el riesgo de errores, como aplicar transformaciones solo a parte de los datos o en un orden incorrecto.

2. Validación Cruzada y Búsqueda de Hiperparámetros: Cuando realizas validación cruzada o búsqueda de hiperparámetros (por ejemplo, usando GridSearchCV o RandomizedSearchCV), un Pipeline asegura que el preprocesamiento se aplique dentro de cada iteración del proceso de validación cruzada, evitando así el riesgo de contaminación de datos entre el entrenamiento y el conjunto de validación.

3. Consistencia en el Preprocesamiento: Para garantizar que el preprocesamiento aplicado a los datos de entrenamiento sea exactamente el mismo que se aplica a los datos de prueba o a nuevos datos antes de hacer predicciones. Esto es crucial para el rendimiento del modelo



Pipeline en Machine learning

Cuando NO usarlo



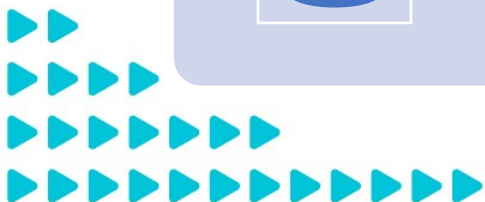
Procesos Altamente Personalizados: Si cada muestra de tus datos necesita un tipo de preprocesamiento muy específico y altamente personalizado que no se puede generalizar bien a través de los transformadores estándar de scikit-learn o que requiere decisiones en tiempo de ejecución basadas en la muestra, integrar esto en un Pipeline puede ser complicado y poco práctico.



Exploración de Datos Detallada: Durante la fase inicial de exploración y análisis de datos, donde estás más interesado en visualizar y entender los datos que en preparar un flujo de trabajo de modelado automatizado. En esta etapa, el uso de Pipeline puede ser prematuro.



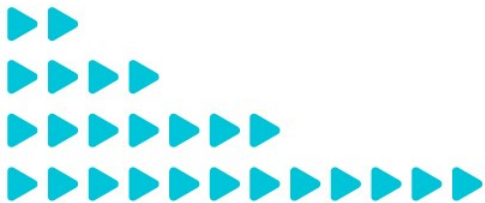
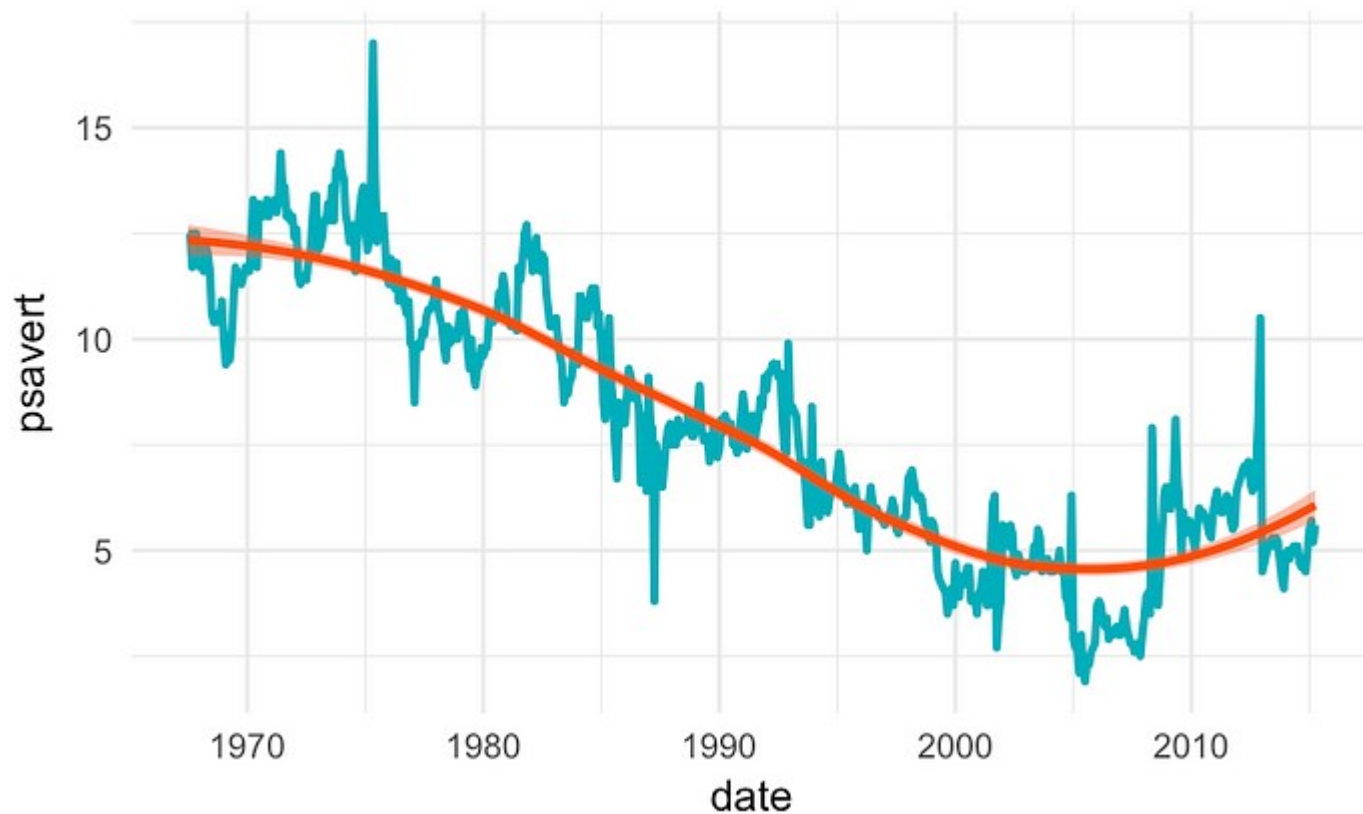
Modelos que Requieren Entradas Diferentes: Si estás utilizando modelos que requieren diferentes subconjuntos de características o transformaciones específicas para diferentes partes del modelo (por ejemplo, partes de un modelo de aprendizaje ensamblado que operan en diferentes vistas de los datos), puede ser difícil encapsular esto limpiamente en un único Pipeline. Aunque ColumnTransformer y FeatureUnion pueden ayudar, en casos muy complejos, gestionar los flujos de trabajo manualmente podría ofrecer más flexibilidad.



Series de tiempo

Conceptos

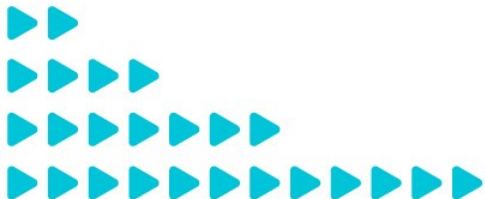
Las series de tiempo en machine learning se refieren al análisis de conjuntos de datos que se recopilan o se registran en intervalos de tiempo regulares. Lo que hace únicas a las series de tiempo es que el orden temporal de los datos es crucial y proporciona información significativa para el análisis y la predicción.



Preprocesamiento de datos

Clases

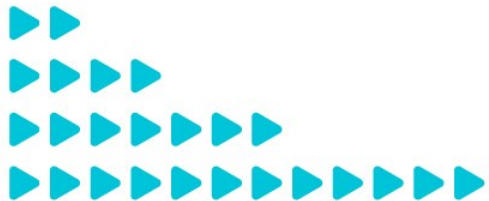
Los datos de series temporales a menudo contienen valores faltantes, valores atípicos y datos ruidosos que pueden afectar el rendimiento del modelo. Por tanto, es fundamental preprocesar los datos antes de entrenar el modelo. Algunas técnicas comunes para el preprocesamiento de datos incluyen la imputación, el manejo de valores atípicos y el escalado.



Extracción de características

Clases

La extracción de características implica extraer información relevante de los datos de series de tiempo que se pueden utilizar para modelar. Algunas técnicas populares de extracción de características incluyen Estadísticas rodantes, Transformada de Fourier y Transformada Wavelet.



Modelado

Clases



UNA VEZ QUE LOS DATOS SE HAN PREPROCESADO Y SE HAN EXTRAÍDO LAS CARACTERÍSTICAS, SE PUEDEN UTILIZAR PARA MODELAR.



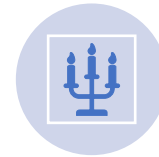
ALGUNOS MODELOS POPULARES PARA DATOS DE SERIES TEMPORALES INCLUYEN



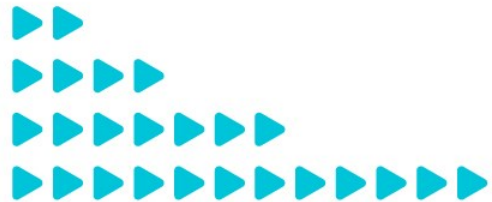
ARIMA: MEDIA MÓVIL INTEGRADA AUTORREGRESIVA (ARIMA),



LSTM: MEMORIA LARGA A CORTO PLAZO (LSTM) Y



PROPHET.



Elección del método

Conceptos

Técnica	Tipo	Descripción	Aplicaciones
LSTM	Red neuronal	Una arquitectura de red neuronal recurrente diseñada para aprender dependencias a largo plazo. Es especialmente útil para secuencias temporales.	Predicción de series temporales, procesamiento de lenguaje natural, reconocimiento de voz.
Regresión Logística	Estadística/Machine Learning	Un modelo de regresión utilizado para predecir la probabilidad de una variable categórica binaria.	Clasificación binaria, como determinar si un email es spam o no.
Regresión por Random Forest	Machine Learning	Un método de ensamble que utiliza múltiples árboles de decisión para realizar predicciones.	Predicción de variables continuas (regresión) y clasificación.
ARIMA	Series temporales	Modelo lineal que utiliza diferenciación para hacer la serie temporal estacionaria y luego aplica un modelo autoregresivo y de promedio móvil.	Predicción de datos de series temporales en finanzas, economía, y otros campos.
ARIMAX	Series temporales	Extensión del modelo ARIMA que incluye variables exógenas para mejorar las predicciones.	Modelado de series temporales donde se conocen factores externos que influyen en la variable de interés.



Técnicas de evaluación de precisión en ML

O análisis de error

Métrica	Definición	Uso
R (Coeficiente de Correlación de Pearson)	Mide la correlación lineal entre dos variables. Varía entre -1 y 1, donde 1 significa una correlación positiva perfecta, -1 una correlación negativa perfecta, y 0 ninguna correlación.	Utilizado para evaluar la fuerza y la dirección de la relación lineal entre variables.
MSE (Mean Squared Error)	Es el promedio de los cuadrados de los errores. El error es la diferencia entre los valores observados y los predichos.	Utilizado para medir la calidad de un estimador; es siempre no negativo, y valores más bajos indican un mejor ajuste.
RMSE (Root Mean Squared Error)	Es la raíz cuadrada del MSE. Proporciona una medida de la magnitud del error en las mismas unidades que la variable de respuesta.	Utilizado para medir la diferencia entre valores predichos por un modelo y los valores observados.
MAE (Mean Absolute Error)	Es el promedio de los valores absolutos de los errores.	Utilizado para medir cuánto difieren las predicciones de un modelo de los valores reales. Es menos sensible a los outliers que el MSE.
R² (Coeficiente de Determinación)	Proporciona una medida de cuánto de la variabilidad de una variable dependiente es explicada por el modelo. Varía entre 0 y 1.	Utilizado para evaluar la bondad de ajuste de un modelo lineal. Cuanto más cercano a 1, mejor explica el modelo la variabilidad de los datos.

